

A course-creator SaaS is a **real product**, not a toy.

Everything below is a **practical design roadmap**, not theory. It's opinionated and optimized for *shippable* results, not academic perfection.

[Inference]

Tech Stack Recap

- **Frontend:** React + JavaScript (inside Next.js)
- **Backend:** Next.js API routes
- **Auth:** Firebase Authentication
- **Database + Storage:** Supabase (Postgres + optional file storage) [Inference]
- **Payments:** Stripe (subscriptions)
- **Analytics/Dashboard:** Chart.js

I'll break this into **phases**. You can treat each phase as a mini-milestone.

Phase 0 – Core Domain & Data Modeling [Inference]

Goal: Know exactly what you're building before touching code.

0.1 Define actors

- **Creator** – uploads courses, manages content, sees revenue.
- **Student** – enrolls, watches content, takes quizzes.
- **Platform Admin (optional)** – monitors everything.

0.2 Define core entities

You'll map these to Supabase tables:

- **users** (app-level user, linked to Firebase UID)
- **creators** (profile + settings)
- **students** (profile + settings)
- **courses** (title, description, price, visibility, creator_id)

- `modules` (course sections)
- `lessons` (video/content per module)
- `quizzes`, `questions`, `answers`, `quiz_attempts`
- `enrollments` (student ↔ course)
- `subscriptions` (creator's plan, Stripe customer + subscription id)
- `progress` (per student per lesson)

Write this as a **schema draft** on paper/Notion first. Then translate to SQL later.

Deliverable for this phase:

- ERD (even a hand-sketched one)
 - List of all API endpoints you'll eventually need (e.g. `/api/courses`, `/api/enrollments`, etc.)
-

Phase 1 – Project Setup & Integration Skeleton [Inference]

Goal: Get a running Next.js app wired with Firebase, Supabase, Stripe keys.

1.1 Create Next.js app

```
npx create-next-app@latest course-saas --use-npm --js
```

Setup:

- `/app` or `/pages` (pick one and stick with it)
- Global layout with navbar + auth-aware UI

1.2 Configure Firebase Auth

- Create Firebase project
- Enable Email/Password and Google Sign-In

Install SDK:

```
npm install firebase
```

-
- Initialize client-side Firebase in a `firebaseClient.js` file.
- Implement:
 - `signUp(email, password)`
 - `signIn(email, password)`
 - `signOut()`

Add:

- **AuthContext** using React Context to store `currentUser` (Firebase user).
- Protect routes using a simple HOC or middleware:
 - `/creator/*` – require logged-in creator
 - `/student/*` – require logged-in student

1.3 Configure Supabase

- Create Supabase project
- Get Postgres connection + API keys

Install:

```
npm install @supabase/supabase-js
```

-
- Create a server-side Supabase client (for API routes) and optionally a client-side one (for basic reads).
Access via environment variables.

1.4 Wire Firebase User ↔ Supabase User

- On first login:
 - Check if Firebase UID exists in `users` table.
 - If not, create:

- `users` row with `firebase_uid`, email, role (`creator` or `student`).
- Provide a simple onboarding screen:
 - “I want to: [Create Courses] or [Take Courses]” → sets the role.

Deliverable:

- You can sign up, sign in, sign out.
 - A row appears in Supabase `users` table upon first login.
-

Phase 2 – Database Schema & Core Backend API [Inference]

Goal: Solid DB and basic CRUD endpoints.

2.1 Create Supabase tables (minimum version)

Tables:

1. `users`
 - `id` (PK)
 - `firebase_uid` (unique)
 - `email`
 - `role` (`creator` | `student`)
 - timestamps
2. `creators`
 - `id` (PK, FK to `users.id`)
 - `display_name`
 - `bio`
 - `stripe_customer_id` (nullable)
 - `current_plan` (e.g. "free" | "pro")
3. `courses`

- `id`
- `creator_id` (FK)
- `title`
- `description`
- `thumbnail_url`
- `slug`
- `is_published` (boolean)
- `price` (for student sale, if you support per-course payment later) [Inference]

4. `modules`

- `id`
- `course_id`
- `title`
- `order_index`

5. `lessons`

- `id`
- `module_id`
- `title`
- `content_type` (video, text, pdf, etc.)
- `content_url` or `content_body`
- `order_index`

6. `enrollments`

- `id`
- `course_id`
- `student_id`
- `created_at`

7. progress

- id
- student_id
- lesson_id
- completed_at

Define **minimal** indexes (on `creator_id`, `course_id`, etc.).

2.2 Implement Next.js API routes

Inside `/pages/api` or `/app/api`:

- `/api/me` – returns user profile from Supabase using Firebase UID.
- `/api/courses` (GET, POST)
- `/api/courses/[id]` (GET, PUT, DELETE)
- `/api/courses/[id]/modules` (GET, POST)
- `/api/modules/[id]/lessons` (GET, POST)
- `/api/enrollments` (POST)
- `/api/progress` (POST)

All API routes should:

- Get Firebase token from headers or cookies.
- Verify it server-side.
- Map to `users` row in Supabase.
- Enforce basic authorization (creator can only modify own courses).

Deliverable:

- You can hit APIs from Postman or browser and see proper JSON responses.
-

Phase 3 – Creator Dashboard (Course Builder) [Inference]

Goal: Let creators create + manage courses from the UI.

3.1 Creator layout

- Route: `/creator/dashboard`
- Show:
 - List of courses with stats (students enrolled, published status)
 - Button: “Create new course”

3.2 Course creation flow

Pages:

- `/creator/courses/new`
- `/creator/courses/[id]/edit`

Features:

- Create/edit title, description, thumbnail upload (use Supabase storage).
- Save via `/api/courses` POST/PUT.
- Toggle `is_published`.

3.3 Module & lesson builder

On `/creator/courses/[id]/edit`:

- Section to manage modules:
 - Add module
 - Reorder modules (drag & drop later, simple buttons first)
- Section to manage lessons in each module:
 - Add lesson (title, type, content_url/body)
 - Reorder lessons

Store everything in Supabase via the API routes.

Deliverable:

- A creator can log in and fully define a course (structure + content references).
-

Phase 4 – Student Portal & Course Consumption [Inference]

Goal: Students can enroll and consume content, and their progress is tracked.

4.1 Public course catalog

- Route: `/courses`
- Show only `is_published = true` courses.
- Each item → `/courses/[slug]`

4.2 Course detail page

- Route: `/courses/[slug]`
- Show:
 - Title, description, creator info
 - “Enroll” button (if logged in as student)
- On enroll:
 - POST `/api/enrollments`
 - Redirect to `/learn/[courseId]`

4.3 Learning experience

- Route: `/learn/[courseId]`
- Show:
 - Left sidebar: modules + lessons
 - Main panel: content of selected lesson
- When student marks lesson complete:

- POST `/api/progress` → insert or update `progress` row.

4.4 Progress logic

- For each course:
 - % complete = completed lessons / total lessons
- Store computed stats in memory or query from Supabase.

Deliverable:

- Student can go from “discover course” → “enroll” → “learn” → “completion tracked.”
-

Phase 5 – Stripe Subscriptions for Creators [Inference]

Goal: Creators pay monthly/yearly to use the platform.

5.1 Stripe setup

- Create Stripe account
- Define:
 - Product: `Course SaaS`
 - Prices: e.g. `starter_monthly`, `pro_monthly`, etc.

5.2 Backend: checkout session endpoint

- POST `/api/billing/create-checkout-session`
 - Requires authenticated creator
 - Creates or reuses Stripe customer for this creator (`stripe_customer_id` in `creators` table)
 - `mode: 'subscription'`
 - `success_url` and `cancel_url` → front-end routes (e.g. `/creator/billing?status=success`)

5.3 Frontend: billing page

- Route: `/creator/billing`
- If not subscribed:
 - “Upgrade to Pro” → hits `/api/billing/create-checkout-session` → redirect to Stripe Checkout.
- If subscribed:
 - Show plan, next billing date, manage-subscription link.

5.4 Webhooks: sync Stripe → Supabase

- Create `/api/webhooks/stripe` (Route Handler / API route).
- Handle events:
 - `checkout.session.completed`
 - `customer.subscription.updated`
 - `customer.subscription.deleted`
- Update `creators.current_plan`, maybe store `subscription_status`, `current_period_end`.

Secure this route:

- Validate the Stripe signature with webhook signing secret.

5.5 Enforce plan limits

In key APIs (course creation, student limit, etc.):

- If `current_plan === 'free'`:
 - Limit max courses (e.g. 1 course).
- If `current_plan === 'pro'`:
 - Lift/expand limits.

Deliverable:

- You can create a test creator, upgrade via Stripe Checkout in test mode, and see their plan update in Supabase and UI.
-

Phase 6 – Analytics Dashboard with Chart.js [Inference]

Goal: Give creators meaningful insight: revenue, enrollments, course performance.

6.1 Define metrics

Per creator, you want:

- Total enrolled students
- Enrollments per course
- Course completion rate
- MRR (monthly recurring revenue from Stripe subscriptions)
- Active students over time (last 30 days)

6.2 Analytics API endpoints

Examples:

- `/api/analytics/overview`
Returns:
 - `total_students`
 - `total_courses`
 - `total_enrollments`
 - `completion_rate_overall`
- `/api/analytics/enrollments-over-time`
Group `enrollments` by day/week.
- `/api/analytics/revenue`
You can:
 - Either fetch from your own DB if you store invoices/amounts
 - Or pull from Stripe API (server-side) and cache results [Inference]

6.3 Frontend – Chart.js

Install:

```
npm install chart.js react-chartjs-2
```

Create charts:

- Line chart: enrollments over time
- Bar chart: per-course enrollments
- Pie chart: completion rate per course

Example:

```
import { Line } from 'react-chartjs-2';

function EnrollmentsChart({ data }) {
  const chartData = {
    labels: data.map(d => d.date),
    datasets: [
      {
        label: 'Enrollments',
        data: data.map(d => d.count),
      },
    ],
  };

  return <Line data={chartData} />;
}
```

Dashboard route: `/creator/analytics`

Deliverable:

- Creator dashboard shows live charts based on real DB/Stripe data.

Phase 7 – “Extra SaaS Features” Layer [Inference]

Here you pull away from basic CRUD and into “this feels like a real product”.

7.1 Drip content

Add fields:

- `courses.drip_enabled` (boolean)

- `lessons.drip_delay_days` (int, days from enrollment)

Logic:

- When student opens `/learn/[courseId]`:
 - Based on enrollment date, filter lessons whose `drip_delay_days` $\leq$ days since enrollment.
- Optionally run a cron/edge function (Supabase scheduled function) to send “new lesson unlocked” emails. [Inference]

7.2 Basic email notifications

You’ll need an email provider (Resend, SendGrid, etc.) – external to your current stack but realistic for SaaS. [Inference]

- On enrollment:
 - Send “Welcome to the course” email.
- On new drip lesson:
 - Send “New lesson unlocked” email.

7.3 Quizzes (minimal)

Tables:

- `quizzes` (per lesson)
- `questions`
- `options`
- `quiz_attempts`

UI:

- On lesson page, if quiz exists:
 - Show MCQs
 - On submit:
 - Evaluate score
 - Store attempt

- Show result to student

Analytics:

- Add quiz performance to Chart.js dashboard (avg score, attempt count).
-

Phase 8 – Hardening & Deployment [Inference]

Goal: Make it production-shaped, not a hacky local toy.

8.1 Security hygiene

- Verify Firebase tokens server-side on every protected API route.
- Use Supabase Row Level Security (RLS):
 - Ensure user can only access their own data (creator_id or student_id checks).
- Validate all JSON inputs (Zod/Yup or custom checks).

8.2 Performance basics

- Use SWR/React Query on heavy list pages.
- Add pagination / infinite scroll where needed.
- Add indexing in Supabase for frequently queried fields.

8.3 Deployments

- Deploy Next.js app to **Vercel**.
- Configure:
 - Firebase in production mode
 - Supabase production keys & URL
 - Stripe live keys & webhook endpoint

8.4 Monitoring

- Use something like:
 - Vercel analytics

- Supabase logs
- Stripe dashboard
- Add basic logging in API routes.

Deliverable:

- Deployed app with working auth, DB, subscriptions, analytics, and creator/student flows.
-

Rough Build Order (Minimal Path)

If you want a condensed order instead of phases: [Inference]

1. Setup Next.js + Firebase Auth + Supabase wiring.
 2. Implement DB schema in Supabase.
 3. Build Creator Dashboard → Course Builder (CRUD).
 4. Build Student Portal → Enrollment → Learning + Progress.
 5. Integrate Stripe subscriptions for creators + webhooks.
 6. Add Analytics dashboards with Chart.js (enrollments, completion).
 7. Add Drip content + basic emails.
 8. Harden security + deploy to Vercel.
-

This roadmap is enough to build a **paid, multi-tenant SaaS** that you can actually show people. From here, the next level is: refactor parts into reusable modules, add TypeScript, tests, and optional AI features (quiz generation, copywriting, etc.), but that's version 2 territory.

You said:

Complete guide for phase 1 that really helps in building this project step by step without skipping any important work. So that the web app don't crash or show any error.

ChatGPT said:

Thought for 17s

Good, now we're talking about real engineering instead of vibes.

You want **Phase 1** done in a way that doesn't randomly explode with errors. I'll walk you through it **step-by-step**, from `npm create-next-app` to:

- Firebase Auth working
- Supabase wired
- User auto-synced into DB
- Basic protected route
- Minimal error handling so things don't crash every 5 minutes

Everything below assumes **JavaScript + App Router in Next.js**. [Inference]

0. Prerequisites (don't skip)

Install / verify:

```
node -v    # >= 18 [Inference]
npm -v
```

Create a folder where you keep your projects, e.g. `~/projects`.

Have accounts ready:

- Google account (for Firebase + Supabase)
- GitHub (optional but recommended for version control) [Inference]

1. Create the Next.js project

In your terminal:

```
cd ~/projects
npm create-next-app@latest course-saas
cd course-saas
```

When it asks:

- TypeScript? → **No** (you said JS)

- ESLint? → **Yes** [Inference]
- App Router? → **Yes**
- Tailwind? → **Optional**, but I'd say yes, it speeds up UI. [Inference]

Run it:

```
npm run dev
```

Open <http://localhost:3000> – verify the starter page loads.

If it doesn't, fix this **before** continuing. No point wiring Firebase on a broken base.

2. Setup environment variables (critical for not leaking keys)

Create a file:

```
touch .env.local
```

Never commit `.env.local` to git. Next.js ignores it by default; keep it that way.

We'll fill it after we get Firebase & Supabase keys. For now, just know:

- Anything prefixed with `NEXT_PUBLIC_` is available on **client + server**
- Non-prefixed vars are **server-only**

You'll need these later:

```
# Firebase
NEXT_PUBLIC_FIREBASE_API_KEY=...
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=...
NEXT_PUBLIC_FIREBASE_PROJECT_ID=...
NEXT_PUBLIC_FIREBASE_APP_ID=...
```

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=...
NEXT_PUBLIC_SUPABASE_ANON_KEY=...
```

```
# (Later) Stripe keys etc.
```

3. Setup Firebase project & Authentication

1. Go to Firebase Console → “Add project”.
2. Create a **Web app**.
3. Enable **Email/Password** and optionally **Google** in **Authentication** → **Sign-in method**.
4. Copy the web config from Firebase, you'll see something like:

```
const firebaseConfig = {  
  apiKey: "xxx",  
  authDomain: "xxx.firebaseio.com",  
  projectId: "xxx",  
  appId: "xxx",  
};
```

Put those into `.env.local`:

```
NEXT_PUBLIC_FIREBASE_API_KEY=xxx  
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=xxx.firebaseio.com  
NEXT_PUBLIC_FIREBASE_PROJECT_ID=xxx  
NEXT_PUBLIC_FIREBASE_APP_ID=xxx
```

3.1 Install Firebase SDK

```
npm install firebase
```

3.2 Create a Firebase client module

Create `lib/firebaseClient.js`:

```
// lib/firebaseClient.js  
import { initializeApp, getApps, getApp } from "firebase/app";  
import { getAuth } from "firebase/auth";  
  
const firebaseConfig = {  
  apiKey: process.env.NEXT_PUBLIC_FIREBASE_API_KEY,  
  authDomain: process.env.NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN,
```

```

    projectId: process.env.NEXT_PUBLIC_FIREBASE_PROJECT_ID,
    appId: process.env.NEXT_PUBLIC_FIREBASE_APP_ID,
  };

  let app;

  if (!getApps().length) {
    app = initializeApp(firebaseConfig);
  } else {
    app = getApp();
  }

  export const auth = getAuth(app);

```

Why this pattern?

- `getApps()` check prevents “**Firestore app already initialized**” errors in dev (Next.js renders components multiple times in dev). [Inference]

4. Create an Auth Context (centralized auth state)

You **never** want to manually call `onAuthStateChanged` in 10 different components. Wrap it.

Install minimal deps (already have React via Next.js).

Create `contexts/AuthContext.js`:

```

"use client";

import { createContext, useContext, useEffect, useState } from
"react";
import { auth } from "@/lib/firebaseClient";
import {
  onAuthStateChanged,
  signInWithEmailAndPassword,
  createUserWithEmailAndPassword,
  signOut,
} from "firebase/auth";

const AuthContext = createContext(null);

```

```

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true); // important to
avoid flicker
  const [error, setError] = useState(null);

  useEffect(() => {
    // Subscribe to auth state
    const unsubscribe = onAuthStateChanged(auth, async
(firebaseUser) => {
      try {
        setError(null);

        if (firebaseUser) {
          // Optionally get ID token here, or just store
firebaseUser
          const token = await firebaseUser.getIdToken();

          setUser({
            uid: firebaseUser.uid,
            email: firebaseUser.email,
            token,
          });

          // Trigger backend sync later in Phase 1 (to Supabase)
        } else {
          setUser(null);
        }
      } catch (err) {
        console.error("Auth state error:", err);
        setError("Failed to update auth state");
      } finally {
        setLoading(false);
      }
    });

    // Clean up subscription to avoid memory leaks
    return () => unsubscribe();
  }, []);

  const login = async (email, password) => {

```

```
setError(null);
try {
  await signInWithEmailAndPassword(auth, email, password);
} catch (err) {
  console.error("Login error:", err);
  setError(err.message);
  throw err;
}
};

const register = async (email, password) => {
  setError(null);
  try {
    await createUserWithEmailAndPassword(auth, email, password);
  } catch (err) {
    console.error("Register error:", err);
    setError(err.message);
    throw err;
  }
};

const logout = async () => {
  setError(null);
  try {
    await signOut(auth);
  } catch (err) {
    console.error("Logout error:", err);
    setError(err.message);
    throw err;
  }
};

const value = {
  user,
  loading,
  error,
  login,
  register,
  logout,
};
```

```
    return <AuthContext.Provider
value={value}>{children}</AuthContext.Provider>;
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) {
    throw new Error("useAuth must be used within AuthProvider");
  }
  return ctx;
}
```

Key anti-crash things:

- **loading** flag to avoid rendering protected UI before auth is ready.
 - **try/catch** everywhere.
 - **unsubscribe** cleanup.
-

5. Wrap the app with **AuthProvider**

In App Router, edit `app/layout.js`:

```
// app/layout.js
import "../globals.css";
import { AuthProvider } from "@contexts/AuthContext";

export const metadata = {
  title: "Course SaaS",
  description: "Course creator SaaS",
};

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>
        <AuthProvider>{children}</AuthProvider>
      </body>
    </html>
  );
}
```

```
);  
}
```

Now the entire app has access to `useAuth()`.

6. Basic Auth Pages (Login / Register) – with error handling

Create `app/login/page.js`:

```
"use client";  
  
import { useState } from "react";  
import { useAuth } from "@/contexts/AuthContext";  
import { useRouter } from "next/navigation";  
  
export default function LoginPage() {  
  const { login, loading, error, user } = useAuth();  
  const router = useRouter();  
  
  const [email, setEmail] = useState("");  
  const [password, setPassword] = useState("");  
  const [submitting, setSubmitting] = useState(false);  
  const [localError, setLocalError] = useState(null);  
  
  // If already logged in, redirect or show info  
  if (user && !loading) {  
    router.push("/dashboard");  
  }  
  
  const handleSubmit = async (e) => {  
    e.preventDefault();  
    setSubmitting(true);  
    setLocalError(null);  
  
    try {  
      await login(email, password);  
      router.push("/dashboard");  
    } catch (err) {  
      setLocalError("Invalid email or password");  
    }  
  }  
}
```

```

    } finally {
      setSubmitting(false);
    }
  };

  return (
    <div style={{ padding: 24 }}>
      <h1>Login</h1>
      {(error || localError) && (
        <p style={{ color: "red" }}>{localError || error}</p>
      )}
      <form onSubmit={handleSubmit}>
        <input
          type="email"
          placeholder="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          required
        />
        <br />
        <input
          type="password"
          placeholder="password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          required
        />
        <br />
        <button type="submit" disabled={submitting || loading}>
          {submitting || loading ? "Logging in..." : "Login"}
        </button>
      </form>
    </div>
  );
}

```

Create `app/register/page.js` similarly using `register()`.

This is enough to confirm Firebase works.

7. Setup Supabase project & client

1. Go to Supabase → New project.
2. Note:
 - Project URL
 - `anon` public key

Add to `.env.local`:

```
NEXT_PUBLIC_SUPABASE_URL=https://xyz.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your_anon_key_here
```

Install client:

```
npm install @supabase/supabase-js
```

Create a client helper: `lib/supabaseClient.js`:

```
import { createClient } from "@supabase/supabase-js";

const supabaseUrl = process.env.NEXT_PUBLIC_SUPABASE_URL;
const supabaseAnonKey = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;

if (!supabaseUrl || !supabaseAnonKey) {
  throw new Error("Missing Supabase env vars");
}

export const supabase = createClient(supabaseUrl, supabaseAnonKey);
```

This throws clearly if env vars are wrong instead of some mysterious runtime crash.

8. Create the `users` table in Supabase

In Supabase SQL editor, run:

```
create table if not exists users (
  id uuid primary key default gen_random_uuid(),
  firebase_uid text unique not null,
```

```
    email text not null,  
    role text check (role in ('creator', 'student')) default  
'student',  
    created_at timestamptz default now()  
);
```

[Inference] We're only using `users` in Phase 1, the rest of schema comes later.

For now, **disable Row Level Security (RLS)** while setting things up to avoid confusion, then later you turn it on and write policies. [Inference]

9. API route to sync Firebase user → Supabase

We want:

- Whenever auth state changes and the user logs in, we call an API route that:
 - Verifies their Firebase token
 - Ensures a row exists in `users`
 - Returns their app profile

9.1 Install Firebase Admin for server-side verification

```
npm install firebase-admin
```

Create `lib/firebaseAdmin.js`:

```
import * as admin from "firebase-admin";  
  
let app;  
  
if (!admin.apps.length) {  
  app = admin.initializeApp({  
    credential: admin.credential.applicationDefault(),  
  });  
} else {  
  app = admin.app();  
}  
  
export const adminAuth = admin.auth();
```

BUT: `applicationDefault()` requires credentials. Safer for now: [Inference]

Instead, use service account JSON via env var. In Firebase → Project settings → Service accounts → “Generate new private key”.

Save the JSON content into `.env.local` as a single string (careful with quotes), e.g.:

```
FIREBASE_SERVICE_ACCOUNT_JSON={
  "type": "...",
  "project_id": "...",
  ...
}
```

Update `firebaseAdmin.js`:

```
import * as admin from "firebase-admin";

let app;

if (!admin.apps.length) {
  const serviceAccount =
    JSON.parse(process.env.FIREBASE_SERVICE_ACCOUNT_JSON);

  app = admin.initializeApp({
    credential: admin.credential.cert(serviceAccount),
  });
} else {
  app = admin.app();
}

export const adminAuth = admin.auth();
```

9.2 Create `/api/auth/sync` route

App Router version: `app/api/auth/sync/route.js`:

```
import { NextResponse } from "next/server";
import { adminAuth } from "@/lib/firebaseAdmin";
import { supabase } from "@/lib/supabaseClient";

export async function POST(req) {
```

```

try {
  const authHeader = req.headers.get("authorization");

  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return NextResponse.json({ error: "Missing token" }, { status:
401 });
  }

  const idToken = authHeader.split("Bearer ")[1];

  // Verify token with Firebase Admin
  const decoded = await adminAuth.verifyIdToken(idToken);
  const { uid, email } = decoded;

  if (!uid || !email) {
    return NextResponse.json(
      { error: "Invalid token payload" },
      { status: 400 }
    );
  }

  // Ensure user exists in Supabase
  const { data: existingUser, error: fetchError } = await supabase
    .from("users")
    .select("*")
    .eq("firebase_uid", uid)
    .single();

  if (fetchError && fetchError.code !== "PGRST116") {
    // PGRST116 -> no rows found
    console.error("Fetch user error:", fetchError);
    return NextResponse.json(
      { error: "Failed to fetch user" },
      { status: 500 }
    );
  }

  let userRecord = existingUser;

  if (!existingUser) {
    const { data: newUser, error: insertError } = await supabase
      .from("users")

```

```

        .insert({
            firebase_uid: uid,
            email,
            role: "student", // default role for now
        })
        .select()
        .single();

    if (insertError) {
        console.error("Insert user error:", insertError);
        return NextResponse.json(
            { error: "Failed to create user" },
            { status: 500 }
        );
    }

    userRecord = newUser;
}

return NextResponse.json({ user: userRecord });
} catch (err) {
    console.error("Sync error:", err);
    return NextResponse.json(
        { error: "Internal server error" },
        { status: 500 }
    );
}
}
}

```

Anti-crash details:

- Check for auth header.
- Gracefully handle “no user found” vs real DB error.
- Log errors to console **but don't leak stack traces to client**.

9.3 Call `/api/auth/sync` from `AuthContext` when user logs in

Update `AuthProvider`'s `onAuthStateChanged` part:

```
useEffect(() => {
```

```
const unsubscribe = onAuthStateChanged(auth, async (firebaseUser)
=> {
  try {
    setError(null);

    if (firebaseUser) {
      const token = await firebaseUser.getIdToken();

      // Call backend sync
      const res = await fetch("/api/auth/sync", {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
          Authorization: `Bearer ${token}`,
        },
      });

      if (!res.ok) {
        console.error("Failed to sync user", await res.text());
        setError("Failed to sync user with backend");
      }

      const data = await res.json();

      setUser({
        uid: firebaseUser.uid,
        email: firebaseUser.email,
        token,
        profile: data.user || null,
      });
    } else {
      setUser(null);
    }
  } catch (err) {
    console.error("Auth state error:", err);
    setError("Failed to update auth state");
  } finally {
    setLoading(false);
  }
});

return () => unsubscribe();
```

```
}, []);
```

Now:

- Firebase handles identity
 - Supabase handles app-level data
 - User is auto-created in Supabase on first login
-

10. Simple protected dashboard route

Create `app/dashboard/page.js`:

```
"use client";

import { useAuth } from "@/contexts/AuthContext";
import { useRouter } from "next/navigation";
import { useEffect } from "react";

export default function DashboardPage() {
  const { user, loading, logout } = useAuth();
  const router = useRouter();

  useEffect(() => {
    if (!loading && !user) {
      router.push("/login");
    }
  }, [loading, user, router]);

  if (loading || !user) {
    return <p>Loading...</p>;
  }

  return (
    <div style={{ padding: 24 }}>
      <h1>Dashboard</h1>
      <p>Logged in as {user.email}</p>
      <button onClick={logout}>Logout</button>
    </div>
  );
}
```

```
}
```

This pattern avoids:

- Rendering dashboard when `user` is still `null` but `loading = true`.
 - Infinite redirects.
 - Crashes from `user.email` when `user` is `null`.
-

11. Home page: logged-in aware

`app/page.js`:

```
"use client";

import { useAuth } from "@/contexts/AuthContext";
import { useRouter } from "next/navigation";
import { useEffect } from "react";

export default function HomePage() {
  const { user, loading } = useAuth();
  const router = useRouter();

  useEffect(() => {
    if (!loading && user) {
      router.push("/dashboard");
    }
  }, [loading, user, router]);

  return (
    <main style={{ padding: 24 }}>
      <h1>Course SaaS</h1>
      {!user && !loading && (
        <>
          <a href="/login">Login</a> | <a
href="/register">Register</a>
        </>
      )}
    </main>
  );
}
```


}

12. Minimal “no stupid errors” checklist for Phase 1

Before you move to Phase 2, confirm each of these:

1. `npm run dev` shows **no red errors** in terminal.
2. `.env.local` exists and contains **all Firebase + Supabase vars**.
3. `http://localhost:3000/login`:
 - You can register a new user.
 - User appears in **Firebase Authentication**.
 - A row appears in Supabase `users` table with matching `firebase_uid`.
4. `http://localhost:3000/dashboard`:
 - Redirects to `/login` if logged out.
 - Loads correctly when logged in.
5. No console spam like:
 - “Firebase app already initialized”
 - “Missing Supabase env vars”
 - “Auth context used outside provider”

If any of these fail, fix them **now**. Don’t pile new features onto a broken base. [Inference]

This is a clean Phase 1 foundation:

- Next.js app bootstrapped
- Firebase auth integrated and centralized
- Supabase wired and synced with Firebase user
- Protected route example
- Error handling in both frontend and backend

Next logical move in Phase 2 will be: expand Supabase schema, define proper API routes for courses/modules/lessons, and start building the creator dashboard. That's where the SaaS starts to look like an actual product instead of a glorified login screen.

You said:

Complete guide for phase 2 that really helps in buiding this project step by step without skipping any important work. So that the web app don't crash or show any error.

ChatGPT said:

Thought for 22s

Alright, Phase 2 is where this stops being a "login demo" and starts becoming an actual SaaS backend.

Goal here:

- Solid Supabase schema
- Clean Next.js API routes
- Centralized auth + error handling
So that you **don't** end up with random 500s, "cannot read property of undefined", and DB chaos.

I'll go in **strict order**. Follow this and you'll have a stable backend foundation.

0. Before Phase 2 – sanity check [Inference]

Do NOT start if any of this is broken:

- `npm run dev` works.
- Login, register, logout via Firebase works.
- `/api/auth/sync` creates a `users` row in Supabase.
- `/dashboard` redirects to `/login` when logged out and works when logged in.

If any of that is not stable, fix it first. Stacking features on a shaky base is how you get mystery bugs.

1. Extend Supabase Schema (Core Tables) [Inference]

You already have `users`. Now add the rest.

Go to **Supabase → SQL Editor**, and run this carefully.

1.1 creators + students

```
create table if not exists creators (  
  id uuid primary key default gen_random_uuid(),  
  user_id uuid not null references users(id) on delete cascade,  
  display_name text,  
  bio text,  
  stripe_customer_id text,  
  current_plan text default 'free',  
  created_at timestamptz default now()  
);
```

```
create unique index if not exists creators_user_id_idx  
on creators(user_id);
```

```
create table if not exists students (  
  id uuid primary key default gen_random_uuid(),  
  user_id uuid not null references users(id) on delete cascade,  
  display_name text,  
  created_at timestamptz default now()  
);
```

```
create unique index if not exists students_user_id_idx  
on students(user_id);
```

- **creators/students** are separate so later you can give one user dual roles if needed.

1.2 courses, modules, lessons

```
create table if not exists courses (  
  id uuid primary key default gen_random_uuid(),  
  creator_id uuid not null references creators(id) on delete  
cascade,  
  title text not null,  
  description text,  
  thumbnail_url text,  
  slug text not null unique,  
  is_published boolean default false,  
  price numeric(10,2),  
  created_at timestamptz default now(),  
  updated_at timestamptz default now())
```

```
);
```

```
create index if not exists courses_creator_id_idx  
on courses(creator_id);
```

```
create table if not exists modules (  
  id uuid primary key default gen_random_uuid(),  
  course_id uuid not null references courses(id) on delete cascade,  
  title text not null,  
  order_index int not null default 0,  
  created_at timestamptz default now()  
);
```

```
create index if not exists modules_course_id_idx  
on modules(course_id);
```

```
create table if not exists lessons (  
  id uuid primary key default gen_random_uuid(),  
  module_id uuid not null references modules(id) on delete cascade,  
  title text not null,  
  content_type text check (content_type in ('video','text','pdf'))  
default 'video',  
  content_url text,  
  content_body text,  
  order_index int not null default 0,  
  created_at timestamptz default now()  
);
```

```
create index if not exists lessons_module_id_idx  
on lessons(module_id);
```

Notes [Inference]:

- `slug` unique → you can route by `/courses/[slug]` later.
- `order_index` lets you sort modules/lessons without insanity.

1.3 enrollments + progress

```
create table if not exists enrollments (  
  id uuid primary key default gen_random_uuid(),  
  course_id uuid not null references courses(id) on delete cascade,
```

```

    student_id uuid not null references students(id) on delete
cascade,
    created_at timestamptz default now(),
    unique (course_id, student_id)
);

create index if not exists enrollments_course_id_idx
on enrollments(course_id);

create index if not exists enrollments_student_id_idx
on enrollments(student_id);

create table if not exists progress (
    id uuid primary key default gen_random_uuid(),
    student_id uuid not null references students(id) on delete
cascade,
    lesson_id uuid not null references lessons(id) on delete cascade,
    completed_at timestamptz default now(),
    unique (student_id, lesson_id)
);

create index if not exists progress_student_id_idx
on progress(student_id);

create index if not exists progress_lesson_id_idx
on progress(lesson_id);

```

RLS: For now, keep RLS OFF while wiring logic. We lock it down in a later phase. Turning it on now without proper policies = “why is everything 401”.

2. Centralized API Auth Helper [Inference]

Stop copying “get user from token” logic into every route. One mistake and you’ve got security holes.

2.1 Server Supabase client

Create `lib/supabaseServer.js`:

```
import { createClient } from "@supabase/supabase-js";
```

```

const supabaseUrl = process.env.NEXT_PUBLIC_SUPABASE_URL;
const supabaseAnonKey = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;

if (!supabaseUrl || !supabaseAnonKey) {
  throw new Error("Missing Supabase env vars");
}

export function getSupabaseServerClient() {
  return createClient(supabaseUrl, supabaseAnonKey);
}

```

You already had `supabaseClient.js`, but that's fine for client-side reads. For API routes, use this helper so we can swap keys later if needed.

2.2 Auth helper: decode token + fetch app user

Create `lib/apiAuth.js`:

```

import { adminAuth } from "../firebaseAdmin";
import { getSupabaseServerClient } from "../supabaseServer";

export async function getAuthUserFromRequest(req) {
  const supabase = getSupabaseServerClient();

  const authHeader = req.headers.get("authorization");

  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return { error: "Missing or invalid Authorization header",
status: 401 };
  }

  const idToken = authHeader.split("Bearer ")[1];

  try {
    const decoded = await adminAuth.verifyIdToken(idToken);
    const { uid, email } = decoded;

    if (!uid || !email) {
      return { error: "Invalid token payload", status: 400 };
    }

    // Fetch user from Supabase

```

```

const { data: user, error: userError } = await supabase
  .from("users")
  .select("*")
  .eq("firebase_uid", uid)
  .single();

if (userError) {
  console.error("Supabase user fetch error:", userError);
  return { error: "User not found", status: 401 };
}

return { user, supabase };
} catch (err) {
  console.error("Auth verification error:", err);
  return { error: "Invalid or expired token", status: 401 };
}
}

```

Pattern:

- Returns { `user`, `supabase` } on success.
- Returns { `error`, `status` } on failure.
- No throwing unless something really catastrophic happens.

You'll reuse this in **every** protected API route.

3. `/api/me` – Test route for auth + user data [Inference]

This is your “ping” route to confirm auth works correctly server-side.

Create: `app/api/me/route.js`:

```

import { NextResponse } from "next/server";
import { getUserFromRequest } from "@/lib/apiAuth";
import { getSupabaseServerClient } from "@/lib/supabaseServer";

export async function GET(req) {
  try {

```

```

    const { user, error, status, supabase } = await
getAuthUserFromRequest(req);

    if (error) {
        return NextResponse.json({ error }, { status });
    }

    // Get creator + student profiles if exist
    const [{ data: creator }, { data: student }] = await
Promise.all([
        supabase
            .from("creators")
            .select("*")
            .eq("user_id", user.id)
            .maybeSingle(),
        supabase
            .from("students")
            .select("*")
            .eq("user_id", user.id)
            .maybeSingle(),
    ]);

    return NextResponse.json({
        user,
        creator,
        student,
    });
} catch (err) {
    console.error("GET /api/me error:", err);
    return NextResponse.json(
        { error: "Internal server error" },
        { status: 500 }
    );
}
}

```

Use `.maybeSingle()` in newer Supabase client versions, or use `.single()` and handle specific error code; adjust as needed. [Inference]

Now you can:

- log in
- get token on client
- call `/api/me` with `Authorization: Bearer token` and see clean JSON.

If this endpoint is not stable, nothing else will be.

4. Creator bootstrap route (optional but useful) [Inference]

You want a way to convert a `user` into a `creator` (and later, `student`).

Create: `app/api/profile/creator/route.js`:

```
import { NextResponse } from "next/server";
import { getAuthUserFromRequest } from "@/lib/apiAuth";

export async function POST(req) {
  try {
    const { user, error, status, supabase } = await
getAuthUserFromRequest(req);
    if (error) return NextResponse.json({ error }, { status });

    const body = await req.json().catch(() => ({}));
    const displayName = body.displayName || user.email;

    // Check if creator already exists
    const { data: existing, error: existingError } = await supabase
      .from("creators")
      .select("*")
      .eq("user_id", user.id)
      .maybeSingle();

    if (existingError) {
      console.error("Fetch creator error:", existingError);
      return NextResponse.json(
        { error: "Failed to check creator" },
        { status: 500 }
      );
    }
  }
```

```

    if (existing) {
      return NextResponse.json({ creator: existing });
    }

    const { data: creator, error: insertError } = await supabase
      .from("creators")
      .insert({
        user_id: user.id,
        display_name: displayName,
      })
      .select()
      .single();

    if (insertError) {
      console.error("Create creator error:", insertError);
      return NextResponse.json(
        { error: "Failed to create creator profile" },
        { status: 500 }
      );
    }

    return NextResponse.json({ creator });
  } catch (err) {
    console.error("POST /api/profile/creator error:", err);
    return NextResponse.json(
      { error: "Internal server error" },
      { status: 500 }
    );
  }
}

```

This will later be called when user chooses “I’m a creator” in onboarding.

5. **/api/courses** – List + create courses [Inference]

For now:

- **GET** → list courses of current creator

- **POST** → create new course

5.1 Helper: **getCreatorOr401**

Instead of pasting checks everywhere:

```
// lib/creatorHelpers.js
export async function getCreatorOrError(supabase, userId) {
  const { data: creator, error } = await supabase
    .from("creators")
    .select("*")
    .eq("user_id", userId)
    .maybeSingle();

  if (error) {
    console.error("Fetch creator error:", error);
    return { error: "Failed to fetch creator", status: 500 };
  }

  if (!creator) {
    return { error: "Creator profile not found", status: 403 };
  }

  return { creator };
}
```

5.2 Install a validation library (optional but strongly recommended) [Inference]

```
npm install zod
```

We'll use this to avoid trash input putting garbage into DB.

5.3 **app/api/courses/route.js**

```
import { NextResponse } from "next/server";
import { z } from "zod";
import { getUserFromRequest } from "@/lib/apiAuth";
import { getCreatorOrError } from "@/lib/creatorHelpers";

const CreateCourseSchema = z.object({
  title: z.string().min(3).max(200),
  description: z.string().max(2000).optional(),
});
```

```
});
```

```
function slugify(text) {  
  return text  
    .toLowerCase()  
    .trim()  
    .replace(/^[^a-z0-9]+/g, "-")  
    .replace(/^-+|-+$/g, "")  
    .slice(0, 60);  
}
```

```
export async function GET(req) {  
  try {  
    const { user, error, status, supabase } = await  
getAuthUserFromRequest(req);  
    if (error) return NextResponse.json({ error }, { status });  
  
    const { creator, error: creatorError, status: creatorStatus } =  
      await getCreatorOrError(supabase, user.id);  
    if (creatorError)  
      return NextResponse.json({ error: creatorError }, { status:  
creatorStatus });  
  
    const { data: courses, error: coursesError } = await supabase  
      .from("courses")  
      .select("*")  
      .eq("creator_id", creator.id)  
      .order("created_at", { ascending: false });  
  
    if (coursesError) {  
      console.error("Fetch courses error:", coursesError);  
      return NextResponse.json(  
        { error: "Failed to fetch courses" },  
        { status: 500 }  
      );  
    }  
  
    return NextResponse.json({ courses });  
  } catch (err) {  
    console.error("GET /api/courses error:", err);  
    return NextResponse.json(  
      { error: "Internal server error" },  
      { status: 500 }  
    );  
  }  
}
```

```

        { status: 500 }
    );
}
}

export async function POST(req) {
    try {
        const { user, error, status, supabase } = await
getAuthUserFromRequest(req);
        if (error) return NextResponse.json({ error }, { status });

        const { creator, error: creatorError, status: creatorStatus } =
            await getCreatorOrError(supabase, user.id);
        if (creatorError)
            return NextResponse.json({ error: creatorError }, { status:
creatorStatus });

        const json = await req.json().catch(() => null);
        if (!json) {
            return NextResponse.json(
                { error: "Invalid JSON body" },
                { status: 400 }
            );
        }

        const parsed = CreateCourseSchema.safeParse(json);
        if (!parsed.success) {
            return NextResponse.json(
                { error: "Validation failed", details: parsed.error.format()
},
                { status: 400 }
            );
        }

        const { title, description } = parsed.data;
        const baseSlug = slugify(title);

        // Ensure unique slug by appending suffix if needed
        let slug = baseSlug;
        let suffix = 1;

        while (true) {

```

```

const { data: existing, error: slugError } = await supabase
  .from("courses")
  .select("id")
  .eq("slug", slug)
  .maybeSingle();

if (slugError) {
  console.error("Slug check error:", slugError);
  return NextResponse.json(
    { error: "Failed to validate slug" },
    { status: 500 }
  );
}

if (!existing) break;

slug = `${baseSlug}-${suffix}`;
suffix += 1;
}

const { data: inserted, error: insertError } = await supabase
  .from("courses")
  .insert({
    creator_id: creator.id,
    title,
    description,
    slug,
  })
  .select()
  .single();

if (insertError) {
  console.error("Create course error:", insertError);
  return NextResponse.json(
    { error: "Failed to create course" },
    { status: 500 }
  );
}

return NextResponse.json({ course: inserted }, { status: 201 });
} catch (err) {
  console.error("POST /api/courses error:", err);
}

```

```

    return NextResponse.json(
      { error: "Internal server error" },
      { status: 500 }
    );
  }
}

```

Anti-crash behavior:

- Validates input.
 - Handles bad JSON.
 - Logs DB errors but doesn't expose internals to client.
 - Unique slug logic avoids DB unique constraint exceptions.
-

6. `/api/courses/[id]` – Get / Update / Delete [Inference]

Create folder: `app/api/courses/[id]/route.js`.

```

import { NextResponse } from "next/server";
import { z } from "zod";
import { getAuthUserFromRequest } from "@lib/apiAuth";
import { getCreatorOrError } from "@lib/creatorHelpers";
import { getSupabaseServerClient } from "@lib/supabaseServer";

```

```

const UpdateCourseSchema = z.object({
  title: z.string().min(3).max(200).optional(),
  description: z.string().max(2000).optional(),
  is_published: z.boolean().optional(),
});

```

```

async function getCourseOwnedByCreator(supabase, courseId,
creatorId) {
  const { data: course, error } = await supabase
    .from("courses")
    .select("*")
    .eq("id", courseId)
    .eq("creator_id", creatorId)

```

```

        .maybeSingle());

    if (error) {
        console.error("Fetch course error:", error);
        return { error: "Failed to fetch course", status: 500 };
    }

    if (!course) {
        return { error: "Course not found", status: 404 };
    }

    return { course };
}

export async function GET(req, { params }) {
    try {
        const { id } = params;
        const { user, error, status, supabase } = await
getAuthUserFromRequest(req);
        if (error) return NextResponse.json({ error }, { status });

        const { creator, error: creatorError, status: creatorStatus } =
            await getCreatorOrError(supabase, user.id);
        if (creatorError)
            return NextResponse.json({ error: creatorError }, { status:
creatorStatus });

        const { course, error: courseError, status: courseStatus } =
            await getCourseOwnedByCreator(supabase, id, creator.id);
        if (courseError)
            return NextResponse.json({ error: courseError }, { status:
courseStatus });

        return NextResponse.json({ course });
    } catch (err) {
        console.error("GET /api/courses/[id] error:", err);
        return NextResponse.json(
            { error: "Internal server error" },
            { status: 500 }
        );
    }
}

```



```

export async function PUT(req, { params }) {
  try {
    const { id } = params;
    const { user, error, status, supabase } = await
getAuthUserFromRequest(req);
    if (error) return NextResponse.json({ error }, { status });

    const { creator, error: creatorError, status: creatorStatus } =
      await getCreatorOrError(supabase, user.id);
    if (creatorError)
      return NextResponse.json({ error: creatorError }, { status:
creatorStatus });

    const { course, error: courseError, status: courseStatus } =
      await getCourseOwnedByCreator(supabase, id, creator.id);
    if (courseError)
      return NextResponse.json({ error: courseError }, { status:
courseStatus });

    const json = await req.json().catch(() => null);
    if (!json) {
      return NextResponse.json(
        { error: "Invalid JSON body" },
        { status: 400 }
      );
    }

    const parsed = UpdateCourseSchema.safeParse(json);
    if (!parsed.success) {
      return NextResponse.json(
        { error: "Validation failed", details: parsed.error.format()
},
        { status: 400 }
      );
    }

    const payload = parsed.data;

    const { data: updated, error: updateError } = await supabase
      .from("courses")
      .update({

```

```

        ...payload,
        updated_at: new Date().toISOString(),
    })
    .eq("id", course.id)
    .select()
    .single();

    if (updateError) {
        console.error("Update course error:", updateError);
        return NextResponse.json(
            { error: "Failed to update course" },
            { status: 500 }
        );
    }

    return NextResponse.json({ course: updated });
} catch (err) {
    console.error("PUT /api/courses/[id] error:", err);
    return NextResponse.json(
        { error: "Internal server error" },
        { status: 500 }
    );
}
}

export async function DELETE(req, { params }) {
    try {
        const { id } = params;
        const { user, error, status, supabase } = await
getAuthUserFromRequest(req);
        if (error) return NextResponse.json({ error }, { status });

        const { creator, error: creatorError, status: creatorStatus } =
            await getCreatorOrError(supabase, user.id);
        if (creatorError)
            return NextResponse.json({ error: creatorError }, { status:
creatorStatus });

        const { course, error: courseError, status: courseStatus } =
            await getCourseOwnedByCreator(supabase, id, creator.id);
        if (courseError)

```

```

    return NextResponse.json({ error: courseError }, { status:
courseStatus });

    const { error: deleteError } = await supabase
      .from("courses")
      .delete()
      .eq("id", course.id);

    if (deleteError) {
      console.error("Delete course error:", deleteError);
      return NextResponse.json(
        { error: "Failed to delete course" },
        { status: 500 }
      );
    }

    return NextResponse.json({ success: true });
  } catch (err) {
    console.error("DELETE /api/courses/[id] error:", err);
    return NextResponse.json(
      { error: "Internal server error" },
      { status: 500 }
    );
  }
}

```

You now have stable CRUD for courses scoped to the logged-in creator.

7. Modules & Lessons APIs (core pieces only) [Inference]

You don't need every edge-case now. Get basic create/list.

7.1 `/api/courses/[id]/modules`

`app/api/courses/[id]/modules/route.js`:

```

import { NextResponse } from "next/server";
import { z } from "zod";
import { getAuthUserFromRequest } from "@/lib/apiAuth";
import { getCreatorOrError } from "@/lib/creatorHelpers";

```

```

const ModuleSchema = z.object({
  title: z.string().min(3).max(200),
  order_index: z.number().int().optional(),
});

async function getOwnedCourse(supabase, courseId, creatorId) {
  const { data: course, error } = await supabase
    .from("courses")
    .select("id")
    .eq("id", courseId)
    .eq("creator_id", creatorId)
    .maybeSingle();

  if (error) {
    console.error("Fetch course error:", error);
    return { error: "Failed to fetch course", status: 500 };
  }

  if (!course) return { error: "Course not found", status: 404 };

  return { course };
}

export async function GET(req, { params }) {
  try {
    const { id: courseId } = params;

    const { user, error, status, supabase } = await
    getAuthUserFromRequest(req);
    if (error) return NextResponse.json({ error }, { status });

    const { creator, error: creatorError, status: creatorStatus } =
      await getCreatorOrError(supabase, user.id);
    if (creatorError)
      return NextResponse.json({ error: creatorError }, { status:
      creatorStatus });

    const { course, error: courseError, status: courseStatus } =
      await getOwnedCourse(supabase, courseId, creator.id);
    if (courseError)

```

```

    return NextResponse.json({ error: courseError }, { status:
courseStatus });

    const { data: modules, error: modulesError } = await supabase
      .from("modules")
      .select("*")
      .eq("course_id", course.id)
      .order("order_index", { ascending: true });

    if (modulesError) {
      console.error("Fetch modules error:", modulesError);
      return NextResponse.json(
        { error: "Failed to fetch modules" },
        { status: 500 }
      );
    }

    return NextResponse.json({ modules });
  } catch (err) {
    console.error("GET /api/courses/[id]/modules error:", err);
    return NextResponse.json(
      { error: "Internal server error" },
      { status: 500 }
    );
  }
}

export async function POST(req, { params }) {
  try {
    const { id: courseId } = params;
    const { user, error, status, supabase } = await
getAuthUserFromRequest(req);
    if (error) return NextResponse.json({ error }, { status });

    const { creator, error: creatorError, status: creatorStatus } =
      await getCreatorOrError(supabase, user.id);
    if (creatorError)
      return NextResponse.json({ error: creatorError }, { status:
creatorStatus });

    const { course, error: courseError, status: courseStatus } =
      await getOwnedCourse(supabase, courseId, creator.id);

```

```

    if (courseError)
        return NextResponse.json({ error: courseError }, { status:
courseStatus });

    const json = await req.json().catch(() => null);
    if (!json) {
        return NextResponse.json(
            { error: "Invalid JSON body" },
            { status: 400 }
        );
    }

    const parsed = ModuleSchema.safeParse(json);
    if (!parsed.success) {
        return NextResponse.json(
            { error: "Validation failed", details: parsed.error.format()
},
            { status: 400 }
        );
    }

    const { title, order_index } = parsed.data;

    // If order_index is missing, assign max+1
    let finalOrderIndex = order_index;
    if (finalOrderIndex === undefined) {
        const { data: maxRow, error: maxError } = await supabase
            .from("modules")
            .select("order_index")
            .eq("course_id", course.id)
            .order("order_index", { ascending: false })
            .limit(1)
            .maybeSingle();

        if (maxError && maxError.code !== "PGRST116") {
            console.error("Modules max index error:", maxError);
            return NextResponse.json(
                { error: "Failed to determine order index" },
                { status: 500 }
            );
        }
    }

```

```

    finalOrderIndex = maxRow ? maxRow.order_index + 1 : 0;
  }

  const { data: module, error: insertError } = await supabase
    .from("modules")
    .insert({
      course_id: course.id,
      title,
      order_index: finalOrderIndex,
    })
    .select()
    .single();

  if (insertError) {
    console.error("Create module error:", insertError);
    return NextResponse.json(
      { error: "Failed to create module" },
      { status: 500 }
    );
  }

  return NextResponse.json({ module }, { status: 201 });
} catch (err) {
  console.error("POST /api/courses/[id]/modules error:", err);
  return NextResponse.json(
    { error: "Internal server error" },
    { status: 500 }
  );
}
}

```

7.2 /api/modules/[id]/lessons

app/api/modules/[id]/lessons/route.js (simplified):

```

import { NextResponse } from "next/server";
import { z } from "zod";
import { getAuthUserFromRequest } from "@/lib/apiAuth";
import { getCreatorOrError } from "@/lib/creatorHelpers";

const LessonSchema = z.object({

```

```

    title: z.string().min(3).max(200),
    content_type: z.enum(["video", "text", "pdf"]).optional(),
    content_url: z.string().url().optional(),
    content_body: z.string().optional(),
    order_index: z.number().int().optional(),
  });

export async function GET(req, { params }) {
  try {
    const { id: moduleId } = params;
    const { user, error, status, supabase } = await
getAuthUserFromRequest(req);
    if (error) return NextResponse.json({ error }, { status });

    const { creator, error: creatorError, status: creatorStatus } =
      await getCreatorOrError(supabase, user.id);
    if (creatorError)
      return NextResponse.json({ error: creatorError }, { status:
creatorStatus });

    // Ensure module is owned by this creator via course -> creator
    const { data: moduleRow, error: moduleError } = await supabase
      .from("modules")
      .select("id, course_id, course:courses(creator_id)")
      .eq("id", moduleId)
      .maybeSingle();

    if (moduleError) {
      console.error("Fetch module error:", moduleError);
      return NextResponse.json(
        { error: "Failed to fetch module" },
        { status: 500 }
      );
    }

    if (!moduleRow || moduleRow.course.creator_id !== creator.id) {
      return NextResponse.json({ error: "Module not found" },
{ status: 404 });
    }

    const { data: lessons, error: lessonsError } = await supabase
      .from("lessons")

```



```

        .select("*")
        .eq("module_id", moduleId)
        .order("order_index", { ascending: true });

    if (lessonsError) {
        console.error("Fetch lessons error:", lessonsError);
        return NextResponse.json(
            { error: "Failed to fetch lessons" },
            { status: 500 }
        );
    }

    return NextResponse.json({ lessons });
} catch (err) {
    console.error("GET /api/modules/[id]/lessons error:", err);
    return NextResponse.json(
        { error: "Internal server error" },
        { status: 500 }
    );
}
}

export async function POST(req, { params }) {
    try {
        const { id: moduleId } = params;
        const { user, error, status, supabase } = await
getAuthUserFromRequest(req);
        if (error) return NextResponse.json({ error }, { status });

        const { creator, error: creatorError, status: creatorStatus } =
            await getCreatorOrError(supabase, user.id);
        if (creatorError)
            return NextResponse.json({ error: creatorError }, { status:
creatorStatus });

        const { data: moduleRow, error: moduleError } = await supabase
            .from("modules")
            .select("id, course_id, course:courses(creator_id)")
            .eq("id", moduleId)
            .maybeSingle();

        if (moduleError) {

```

```

        console.error("Fetch module error:", moduleError);
        return NextResponse.json(
            { error: "Failed to fetch module" },
            { status: 500 }
        );
    }

    if (!moduleRow || moduleRow.course.creator_id !== creator.id) {
        return NextResponse.json({ error: "Module not found" },
{ status: 404 });
    }

    const json = await req.json().catch(() => null);
    if (!json) {
        return NextResponse.json(
            { error: "Invalid JSON body" },
            { status: 400 }
        );
    }

    const parsed = LessonSchema.safeParse(json);
    if (!parsed.success) {
        return NextResponse.json(
            { error: "Validation failed", details: parsed.error.format()
},
            { status: 400 }
        );
    }

    let {
        title,
        content_type = "video",
        content_url,
        content_body,
        order_index,
    } = parsed.data;

    // Basic sanity: video/pdf should have URL, text should have
body
    if (
        (content_type === "video" || content_type === "pdf") &&
        !content_url

```

```

) {
  return NextResponse.json(
    { error: "content_url is required for video/pdf lessons" },
    { status: 400 }
  );
}

if (content_type === "text" && !content_body) {
  return NextResponse.json(
    { error: "content_body is required for text lessons" },
    { status: 400 }
  );
}

// Determine order_index if missing
if (order_index === undefined) {
  const { data: maxRow, error: maxError } = await supabase
    .from("lessons")
    .select("order_index")
    .eq("module_id", moduleId)
    .order("order_index", { ascending: false })
    .limit(1)
    .maybeSingle();

  if (maxError && maxError.code !== "PGRST116") {
    console.error("Lessons max index error:", maxError);
    return NextResponse.json(
      { error: "Failed to determine order index" },
      { status: 500 }
    );
  }

  order_index = maxRow ? maxRow.order_index + 1 : 0;
}

const { data: lesson, error: insertError } = await supabase
  .from("lessons")
  .insert({
    module_id: moduleId,
    title,
    content_type,
    content_url,
  })

```

```

        content_body,
        order_index,
    })
    .select()
    .single();

    if (insertError) {
        console.error("Create lesson error:", insertError);
        return NextResponse.json(
            { error: "Failed to create lesson" },
            { status: 500 }
        );
    }

    return NextResponse.json({ lesson }, { status: 201 });
} catch (err) {
    console.error("POST /api/modules/[id]/lessons error:", err);
    return NextResponse.json(
        { error: "Internal server error" },
        { status: 500 }
    );
}
}

```

8. Minimal Enrollment & Progress APIs (just to complete Phase 2) [Inference]

Full student flow comes in later phase, but for completeness:

- `/api/enrollments` POST: current user (student) enrolls in course
- `/api/progress` POST: mark lesson completed

You'll first need a `students` profile creator endpoint similar to `profile/creator` above; then:

```

// app/api/enrollments/route.js - outline only
// 1. getAuthUserFromRequest
// 2. get or create student row
// 3. validate course_id from body

```

```
// 4. insert into enrollments with unique constraint, handle
duplicate (already enrolled) gracefully
```

and

```
// app/api/progress/route.js - outline only
// 1. auth + student
// 2. validate lesson_id
// 3. upsert into progress
```

You can use the same patterns: zod validation, clear errors, no thrown exceptions.

9. Phase 2 “no stupid errors” checklist [Inference]

Before saying Phase 2 is done, verify:

1. Schema

- All tables exist in Supabase.
- You can see them in Table Editor.
- `users` rows are being created from Phase 1 auth.

2. `/api/me`

- Called with valid token → returns `{ user, creator|null, student|null }`.
- Called with missing/invalid token → returns **401**, not 500.

3. Creator profile

- `POST /api/profile/creator` after login → creates a row in `creators`.
- Second call does NOT create duplicate row.

4. `/api/courses`

- `GET` with no creator profile → 403.
- `POST` with bad body → 400 + validation details.
- `POST` with valid body → course created, unique slug, no crash.

5. `/api/courses/[id]`

- `GET` another creator's course id → 404 (not "some DB error").
- `PUT` with invalid JSON → 400.
- `DELETE` removes the course row.

6. Modules & Lessons

- `POST /api/courses/[courseId]/modules` → module created with correct order_index.
- `POST /api/modules/[moduleId]/lessons`:
 - video/pdf with no URL → 400 (your validation works).
 - text with no body → 400.
 - valid payload → lesson created.

If something here returns 500, read your logs. Most likely:

- mis-typed env var
- wrong route path
- Supabase error you didn't handle

Phase 2 after this is not "perfect architecture", but it's **good enough** and safe enough to start Phase 3 (creator UI). You've got:

- Clean DB schema for core domain
- Centralized auth
- Scoped access (creator can only touch their stuff)
- Clear validation and error handling

Next step: wire all this with React pages for course builder, and you'll finally see the structure take shape in the UI instead of just Postman responses.

You said:

Complete guide for phase 3 that really helps in buiding this project step by step without skipping any important work. So that the web app don't crash or show any error.

ChatGPT said:

Thought for 18s

Good. Phase 3 is where you stop poking APIs with Postman and actually give the creator a UI that doesn't fall apart every time `user` is briefly `null`.

I'll walk you through **exactly** what to build:

- Auth-guarded creator area
- Creator onboarding
- API client wrapper (with token)
- Creator dashboard (course list)
- New course page
- Course editor page (modules + lessons manager)

All with **null checks, loading states, and error paths** so you don't get "Cannot read property 'x' of undefined" every second.

Anything marked **[Inference]** is design advice, not something you explicitly specified.

0. Sanity Check Before Phase 3 □ [Inference]

Do not start this if:

- Login / register / logout works.
- AuthContext provides `{ user, loading }` where `user.token` is the Firebase ID token.
- `/api/auth/sync` correctly creates a `users` row.
- `/api/me` works with Authorization header.
- `/api/profile/creator` creates a `creators` row.
- `/api/courses` (GET, POST) works with a bearer token and a creator profile.

If any of these fail, fix backend first. UI cannot save nonsense.

1. App Structure for Creator Area [Inference]

We'll use routes like:

- `/creator/onboarding`
- `/creator/dashboard`
- `/creator/courses/new`
- `/creator/courses/[id]/edit`

Make a folder: `app/creator`.

1.1 Creator layout (optional but clean)

`app/creator/layout.js`:

```
// app/creator/layout.js
import "../globals.css";

export const metadata = {
  title: "Creator Area",
};

export default function CreatorLayout({ children }) {
  return (
    <div style={{ display: "flex", minHeight: "100vh" }}>
      <aside
        style={{
          width: 220,
          padding: 16,
          borderRight: "1px solid #ddd",
        }}
      >
        <h2>Creator</h2>
        <nav style={{ display: "flex", flexDirection: "column", gap:
8 }}>
          <a href="/creator/dashboard">Dashboard</a>
          <a href="/creator/courses/new">New Course</a>
        </nav>
      </aside>
      <main style={{ flex: 1, padding: 24 }}>{children}</main>
    </div>
  );
}
```


This layout wraps all `/creator/*` routes.

2. API Client Wrapper (Front-end) [Inference]

You don't want to manually write `fetch("/api/...")` with token and error handling 500 times.

Create `lib/apiClient.js`:

```
// lib/apiClient.js
"use client";

export async function apiRequest(path, options = {}, token) {
  if (!token) {
    throw new Error("Missing auth token for API request");
  }

  const { method = "GET", body, headers = {} } = options;

  const res = await fetch(path, {
    method,
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
      ...headers,
    },
    body: body ? JSON.stringify(body) : undefined,
  });

  let data = null;
  try {
    data = await res.json();
  } catch (e) {
    // Non-JSON, leave as null but don't crash
  }

  if (!res.ok) {
    const message = data?.error || `Request failed with ${res.status}`;
    const error = new Error(message);
    error.status = res.status;
  }
}
```

```
    error.data = data;
    throw error;
  }

  return data;
}
```

Behavior:

- Throws for non-2xx but with a clean message.
- Doesn't explode if response body is empty or not JSON.

You'll use it in all client-side data fetching.

3. Auth Guard + Creator Guard Hooks [Inference]

You want centralized guards so pages don't crash when `user` is not ready.

3.1 `useRequireAuth`

`hooks/useRequireAuth.js`:

```
"use client";

import { useEffect } from "react";
import { useRouter } from "next/navigation";
import { useAuth } from "@/contexts/AuthContext";

export function useRequireAuth() {
  const { user, loading } = useAuth();
  const router = useRouter();

  useEffect(() => {
    if (!loading && !user) {
      router.push("/login");
    }
  }, [loading, user, router]);

  return { user, loading };
}
```

3.2 useCreatorProfile

We'll fetch `/api/me` once and use it to check if creator profile exists.

`hooks/useCreatorProfile.js`:

```
"use client";

import { useEffect, useState } from "react";
import { useRequireAuth } from "../useRequireAuth";
import { apiRequest } from "@lib/apiClient";

export function useCreatorProfile() {
  const { user, loading: authLoading } = useRequireAuth();
  const [creator, setCreator] = useState(null);
  const [profileLoading, setProfileLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchMe = async () => {
      if (authLoading) return;
      if (!user) return; // redirected already

      try {
        setError(null);
        setProfileLoading(true);
        const data = await apiRequest("/api/me", {}, user.token);
        setCreator(data.creator || null);
      } catch (err) {
        console.error("useCreatorProfile error:", err);
        setError(err.message || "Failed to load creator profile");
      } finally {
        setProfileLoading(false);
      }
    };

    fetchMe();
  }, [authLoading, user]);

  return {
    user,
  };
}
```

```
    creator,  
    loading: authLoading || profileLoading,  
    error,  
  };  
}
```

Now you can call `useCreatorProfile()` in any creator page.

4. Creator Onboarding Page [Inference]

If user is logged in but has no creator profile, we push them here.

4.1 /creator/onboarding

app/creator/onboarding/page.js:

```
"use client";  
  
import { useState, useEffect } from "react";  
import { useRequireAuth } from "@/hooks/useRequireAuth";  
import { apiRequest } from "@/lib/apiClient";  
import { useRouter } from "next/navigation";  
  
export default function CreatorOnboardingPage() {  
  const { user, loading } = useRequireAuth();  
  const router = useRouter();  
  
  const [displayName, setDisplayName] = useState("");  
  const [submitting, setSubmitting] = useState(false);  
  const [error, setError] = useState(null);  
  
  useEffect(() => {  
    if (!loading && user && !displayName) {  
      setDisplayName(user.email || "");  
    }  
  }, [loading, user, displayName]);  
  
  const handleSubmit = async (e) => {  
    e.preventDefault();  
    if (!user) return;
```

```

    setSubmitting(true);
    setError(null);

    try {
      await apiRequest(
        "/api/profile/creator",
        {
          method: "POST",
          body: { displayName },
        },
        user.token
      );

      router.push("/creator/dashboard");
    } catch (err) {
      console.error("Onboarding error:", err);
      setError(err.message || "Failed to create creator profile");
    } finally {
      setSubmitting(false);
    }
  };

  if (loading) return <p>Loading...</p>;
  if (!user) return null; // useRequireAuth already redirects

  return (
    <div>
      <h1>Creator Setup</h1>
      <p>Create your creator profile to start building courses.</p>
      {error && <p style={{ color: "red" }}>{error}</p>}
      <form onSubmit={handleSubmit}>
        <label>
          Display name:
          <br />
          <input
            value={displayName}
            onChange={(e) => setDisplayName(e.target.value)}
            required
          />
        </label>
        <br />
        <button type="submit" disabled={submitting}>

```

```
        {submitting ? "Creating..." : "Continue as Creator"}
      </button>
    </form>
  </div>
);
}
```

5. Creator Dashboard – Course List [Inference]

Now build `/creator/dashboard` that:

- Requires auth
- Requires creator profile (if missing → onboarding)
- Lists courses via `/api/courses`
- Handles loading & error states

`app/creator/dashboard/page.js`:

```
"use client";

import { useEffect, useState } from "react";
import { useCreatorProfile } from "@/hooks/useCreatorProfile";
import { useRouter } from "next/navigation";
import { apiRequest } from "@/lib/apiClient";

export default function CreatorDashboardPage() {
  const { user, creator, loading, error: profileError } =
    useCreatorProfile();
  const router = useRouter();

  const [courses, setCourses] = useState([]);
  const [coursesLoading, setCoursesLoading] = useState(true);
  const [coursesError, setCoursesError] = useState(null);

  useEffect(() => {
    if (loading) return;

    if (!user) return; // redirected
    if (!creator) {
```

```

        router.push("/creator/onboarding");
        return;
    }

    const fetchCourses = async () => {
        try {
            setCoursesError(null);
            setCoursesLoading(true);
            const data = await apiRequest("/api/courses", {},
user.token);
            setCourses(data.courses || []);
        } catch (err) {
            console.error("Fetch courses error:", err);
            setCoursesError(err.message || "Failed to load courses");
        } finally {
            setCoursesLoading(false);
        }
    };

    fetchCourses();
}, [loading, user, creator, router]);

if (loading) return <p>Loading...</p>;
if (!user) return null;
if (!creator) return null; // redirect already triggered

return (
    <div>
        <h1>Creator Dashboard</h1>

        {profileError && <p style={{ color: "red"
}}>{profileError}</p>}

        <button onClick={() => router.push("/creator/courses/new")}>
            + Create New Course
        </button>

        {coursesLoading && <p>Loading courses...</p>}
        {coursesError && <p style={{ color: "red"
}}>{coursesError}</p>}

        {!coursesLoading && !coursesError && courses.length === 0 && (

```

```

        <p>No courses yet. Create your first one.</p>
    )}

    <ul>
        {courses.map((course) => (
            <li key={course.id} style={{ marginTop: 12 }}>
                <strong>{course.title}</strong>
                {course.is_published && (
                    <span style={{ marginLeft: 8, color: "green" }}
>(Published)</span>
                )}
                <div>
                    <button
                        onClick={() =>
                            router.push(`/creator/courses/${course.id}/edit`)
                        }
                    >
                        Edit
                    </button>
                    { /* delete button will come later */ }
                </div>
            </li>
        ))}
    </ul>
</div>
);
}

```

Here you have:

- Guard that sends users without creator profile to onboarding
- Safe rendering (no direct `creator.name` while `creator` is `null`)

6. New Course Page – `/creator/courses/new` [Inference]

Form that calls `POST /api/courses` and then redirects to edit page.

`app/creator/courses/new/page.js`:


```
"use client";

import { useState, useEffect } from "react";
import { useCreatorProfile } from "@/hooks/useCreatorProfile";
import { useRouter } from "next/navigation";
import { apiRequest } from "@/lib/apiClient";

export default function NewCoursePage() {
  const { user, creator, loading } = useCreatorProfile();
  const router = useRouter();

  const [title, setTitle] = useState("");
  const [description, setDescription] = useState("");
  const [submitting, setSubmitting] = useState(false);
  const [error, setError] = useState(null);

  useEffect(() => {
    if (!loading && !user) {
      router.push("/login");
    }
    if (!loading && user && !creator) {
      router.push("/creator/onboarding");
    }
  }, [loading, user, creator, router]);

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!user || !creator) return;

    setSubmitting(true);
    setError(null);

    try {
      const data = await apiRequest(
        "/api/courses",
        {
          method: "POST",
          body: { title, description },
        },
        user.token
      );
    }
  }
}
```

```

        const newCourse = data.course;
        router.push(`/creator/courses/${newCourse.id}/edit`);
    } catch (err) {
        console.error("Create course error:", err);
        setError(err.message || "Failed to create course");
    } finally {
        setSubmitting(false);
    }
};

if (loading || !user || !creator) {
    return <p>Loading...</p>;
}

return (
    <div>
        <h1>Create New Course</h1>
        {error && <p style={{ color: "red" }}>{error}</p>}
        <form onSubmit={handleSubmit}>
            <label>
                Title
                <br />
                <input
                    value={title}
                    onChange={(e) => setTitle(e.target.value)}
                    required
                    minLength={3}
                />
            </label>
            <br />
            <label>
                Description
                <br />
                <textarea
                    value={description}
                    onChange={(e) => setDescription(e.target.value)}
                    rows={4}
                />
            </label>
            <br />
            <button type="submit" disabled={submitting}>
                {submitting ? "Creating..." : "Create & Edit"}
            </button>
        </form>
    </div>
);

```

```
        </button>
      </form>
    </div>
  );
}
```

7. Course Editor Page – Shell [Inference]

Route: `/creator/courses/[id]/edit`

We need:

- Load course details
- Load modules list
- Load lessons per module
- Allow adding modules and lessons
- Toggle publish flag later

Start with loading logic + basic UI, then add actions.

`app/creator/courses/[id]/edit/page.js`:

```
"use client";

import { useEffect, useState, useMemo } from "react";
import { useParams, useRouter } from "next/navigation";
import { useCreatorProfile } from "@/hooks/useCreatorProfile";
import { apiRequest } from "@/lib/apiClient";

export default function EditCoursePage() {
  const params = useParams();
  const courseId = params?.id;
  const router = useRouter();

  const { user, creator, loading: profileLoading } =
    useCreatorProfile();

  const [course, setCourse] = useState(null);
  const [modules, setModules] = useState([]);
```

```
const [lessonsByModule, setLessonsByModule] = useState({});
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);
```

```
// For editing title/description
const [title, setTitle] = useState("");
const [description, setDescription] = useState("");
const [savingCourse, setSavingCourse] = useState(false);
```

```
useEffect(() => {
  if (!courseId) return;
  if (profileLoading) return;

  if (!user) {
    router.push("/login");
    return;
  }
  if (!creator) {
    router.push("/creator/onboarding");
    return;
  }
}
```

```
const loadData = async () => {
  try {
    setError(null);
    setLoading(true);

    // 1. fetch course
    const courseData = await apiRequest(
      `/api/courses/${courseId}`,
      {},
      user.token
    );

    const course = courseData.course;
    setCourse(course);
    setTitle(course.title || "");
    setDescription(course.description || "");

    // 2. fetch modules
    const modulesData = await apiRequest(
      `/api/courses/${courseId}/modules`,
```

```

        {},
        user.token
    );
    const mods = modulesData.modules || [];
    setModules(mods);

    // 3. fetch lessons for each module (sequential for
simplicity)
    const lessonsMap = {};
    for (const m of mods) {
        const lessonsData = await apiRequest(
            `/api/modules/${m.id}/lessons`,
            {},
            user.token
        );
        lessonsMap[m.id] = lessonsData.lessons || [];
    }

    setLessonsByModule(lessonsMap);
} catch (err) {
    console.error("Load edit course error:", err);
    setError(err.message || "Failed to load course editor");
} finally {
    setLoading(false);
}
};

loadData();
}, [courseId, profileLoading, user, creator, router]);

const sortedModules = useMemo(
    () => [...modules].sort((a, b) => a.order_index -
b.order_index),
    [modules]
);

if (profileLoading || loading) {
    return <p>Loading...</p>;
}

if (error) {
    return (

```

```

        <div>
          <p style={{ color: "red" }}>{error}</p>
          <button onClick={() => router.push("/creator/dashboard")}>
            Back to dashboard
          </button>
        </div>
      );
    }

    if (!course) {
      return (
        <div>
          <p>Course not found.</p>
          <button onClick={() => router.push("/creator/dashboard")}>
            Back to dashboard
          </button>
        </div>
      );
    }

    const handleSaveCourse = async (e) => {
      e.preventDefault();
      if (!user) return;
      setSavingCourse(true);
      setError(null);

      try {
        const data = await apiRequest(
          `/api/courses/${courseId}`,
          {
            method: "PUT",
            body: {
              title,
              description,
            },
            user.token
          );

        setCourse(data.course);
      } catch (err) {
        console.error("Save course error:", err);
      }
    };
  }

```

```

        setError(err.message || "Failed to save course");
    } finally {
        setSavingCourse(false);
    }
};

return (
    <div>
        <h1>Edit Course</h1>

        <section style={{ marginBottom: 24 }}>
            <h2>Course Details</h2>
            <form onSubmit={handleSaveCourse}>
                <label>
                    Title
                    <br />
                    <input
                        value={title}
                        onChange={(e) => setTitle(e.target.value)}
                        required
                        minLength={3}
                    />
                </label>
                <br />
                <label>
                    Description
                    <br />
                    <textarea
                        value={description}
                        onChange={(e) => setDescription(e.target.value)}
                        rows={4}
                    />
                </label>
                <br />
                <button type="submit" disabled={savingCourse}>
                    {savingCourse ? "Saving..." : "Save"}
                </button>
            </form>
        </section>

        <ModulesSection
            user={user}

```

```

        courseId={courseId}
        modules={sortedModules}
        setModules={setModules}
        lessonsByModule={lessonsByModule}
        setLessonsByModule={setLessonsByModule}
      />
    </div>
  );
}

```

Notice:

- No direct access to `user/creator` until guards pass.
- Handles 404 / not found scenario gracefully.

8. Modules & Lessons UI Components [Inference]

To avoid a 500-line page, split into a component.

8.1 Modules section

Add in same file (below) or separate file:

```

function ModulesSection({
  user,
  courseId,
  modules,
  setModules,
  lessonsByModule,
  setLessonsByModule,
}) {
  const [newModuleTitle, setNewModuleTitle] = useState("");
  const [creatingModule, setCreatingModule] = useState(false);
  const [error, setError] = useState(null);

  const handleAddModule = async (e) => {
    e.preventDefault();
    if (!newModuleTitle.trim()) return;
    setCreatingModule(true);
    setError(null);
  };

```



```

try {
  const data = await apiRequest(
    `/api/courses/${courseId}/modules`,
    {
      method: "POST",
      body: { title: newModuleTitle.trim() },
    },
    user.token
  );

  const module = data.module;
  setModules((prev) => [...prev, module]);
  setLessonsByModule((prev) => ({ ...prev, [module.id]: [] }));
  setNewModuleTitle("");
} catch (err) {
  console.error("Add module error:", err);
  setError(err.message || "Failed to add module");
} finally {
  setCreatingModule(false);
}
};

return (
  <section>
    <h2>Modules & Lessons</h2>
    {error && <p style={{ color: "red" }}>{error}</p>}

    <form onSubmit={handleAddModule}>
      <input
        placeholder="New module title"
        value={newModuleTitle}
        onChange={(e) => setNewModuleTitle(e.target.value)}
        required
      />
      <button type="submit" disabled={creatingModule}>
        {creatingModule ? "Adding..." : "Add Module"}
      </button>
    </form>

    <div style={{ marginTop: 16 }}>
      {modules.length === 0 && <p>No modules yet.</p>}
```

```

        {modules.map((m) => (
          <ModuleItem
            key={m.id}
            module={m}
            user={user}
            lessons={lessonsByModule[m.id] || []}
            setLessonsByModule={setLessonsByModule}
          />
        ))}
      </div>
    </section>
  );
}

```

8.2 Single module item + lessons

```

function ModuleItem({ module, user, lessons, setLessonsByModule }) {
  const [localLessons, setLocalLessons] = useState(lessons);
  const [newLessonTitle, setNewLessonTitle] = useState("");
  const [newLessonUrl, setNewLessonUrl] = useState("");
  const [creatingLesson, setCreatingLesson] = useState(false);
  const [error, setError] = useState(null);

  useEffect(() => {
    setLocalLessons(lessons);
  }, [lessons]);

  const handleAddLesson = async (e) => {
    e.preventDefault();
    if (!newLessonTitle.trim() || !newLessonUrl.trim()) return;

    setCreatingLesson(true);
    setError(null);

    try {
      const data = await apiRequest(
        `/api/modules/${module.id}/lessons`,
        {
          method: "POST",
          body: {
            title: newLessonTitle.trim(),

```

```

        content_type: "video", // [Inference] defaulting to
video here
        content_url: newLessonUrl.trim(),
    },
    },
    user.token
);

const lesson = data.lesson;
setLocalLessons((prev) => [...prev, lesson]);
setLessonsByModule((prev) => ({
    ...prev,
    [module.id]: [...(prev[module.id] || []), lesson],
})));

setNewLessonTitle("");
setNewLessonUrl("");
} catch (err) {
    console.error("Add lesson error:", err);
    setError(err.message || "Failed to add lesson");
} finally {
    setCreatingLesson(false);
}
};

return (
    <div
        style={{
            border: "1px solid #ddd",
            borderRadius: 4,
            padding: 12,
            marginTop: 12,
        }}
    >
        <h3>{module.title}</h3>

        {error && <p style={{ color: "red" }}>{error}</p>}

        <ul>
            {localLessons.map((lesson) => (
                <li key={lesson.id}>
                    {lesson.title} ({lesson.content_type})

```

```

        </li>
      )})
      {localLessons.length === 0 && <li>No lessons yet.</li>}
    </ul>

    <form onSubmit={handleAddLesson} style={{ marginTop: 8 }}>
      <input
        placeholder="Lesson title"
        value={newLessonTitle}
        onChange={(e) => setNewLessonTitle(e.target.value)}
        required
      />
      <input
        placeholder="Video URL"
        value={newLessonUrl}
        onChange={(e) => setNewLessonUrl(e.target.value)}
        required
      />
      <button type="submit" disabled={creatingLesson}>
        {creatingLesson ? "Adding..." : "Add Lesson"}
      </button>
    </form>
  </div>
);
}

```

This gives you a working, non-crashy course editor:

- Add modules
- Add video lessons per module
- Update course title/description

You can extend later (reordering, deleting, text/pdf, etc.).

9. Front-End Error Hygiene Checklist [Inference]

Here's how you keep it from exploding:

Never assume `user` / `creator` exist

Always gate UI with:

```
if (loading) return <p>Loading...</p>;  
if (!user) { router.push("/login"); return null; }  
if (!creator) { router.push("/creator/onboarding"); return null; }
```

- 1.
 2. **Always catch errors around `apiRequest`**
No naked `await apiRequest(...)` in event handlers without try/catch.
 3. **Avoid calling `router.push` in render**
Always inside `useEffect`, not directly in JSX.
 4. **Control submissions with `submitting` flags**
Disable buttons while awaiting, so users don't double-submit and trigger edge-case 409s.
 5. **Don't assume shape of data from API**
Always default: `data.courses || [], data.modules || []`.
 6. **Don't derive state from props without syncing**
If you keep local copy (like `localLessons`), sync on `useEffect` when prop changes.
-

10. Phase 3 Completion Criteria [Inference]

You're done with Phase 3 if:

- Logged-in user can:
 - Run `/creator/onboarding` once and become a creator.
 - Visit `/creator/dashboard` and see a list of their courses.
 - Create a new course and get redirected to its editor.
 - Edit course title/description without errors.
 - Add modules to a course.
 - Add lessons to modules.
- Invalid token → APIs return clean 401, UI shows a sane error or redirects, not a white screen.
- Deleting a table row in Supabase manually doesn't crash the UI; it shows "not found" instead of throwing.

At this point, your SaaS has:

- Auth
- User model
- Creator model
- Course structure
- A functional creator UI

Next phases are: student flow, enrollment, progress, Stripe subscriptions, and analytics. But the core creator-side brain is already online.

You said:

Complete guide for phase 4 that really helps in buiding this project step by step without skipping any important work. So that the web app don't crash or show any error.

ChatGPT said:

Phase 4 is where your SaaS finally becomes usable for *students* — not just creators.

This is also the phase where inexperienced devs break everything:

- wrong access logic
- missing guards
- race conditions between auth & fetch
- fetching lessons before modules exist
- treating relations incorrectly
- letting students see other creators' courses
- letting students mark progress in courses they never enrolled in

I'll walk you through a **bullet-proof, error-tolerant, production-safe Phase 4**, without skipping any critical step.

PHASE 4 – STUDENT PORTAL & LEARNING ENGINE (FULL GUIDE)

What Phase 4 Delivers

You will implement:

1. **Public course catalog** (`/courses`)
2. **Course detail page** (`/courses/[slug]`)
3. **Enrollment system** (API + frontend)
4. **Student learning view** (`/learn/[courseId]`)
5. **Progress tracking system**
6. **Access control so only enrolled students can view lessons**

Everything must remain stable even when:

- user is not logged in
- course does not exist
- slug mismatches
- creator deletes a module
- lessons return empty results
- user opens weird URLs

This guide prevents all of that.

0. PRE-CHECKLIST (DO NOT SKIP)

If ANY of these are broken, Phase 4 will fail:

- `/api/courses` works
- `/api/courses/[id]` works
- `/api/courses/[id]/modules` works
- `/api/modules/[id]/lessons` works
- AuthContext returns correct `user.token`
- Users can log in and out normally
- Creator UI loads without runtime errors

If not stable → fix before touching Phase 4.

1. PUBLIC COURSE CATALOG

(/courses)

This is a **non-authenticated** page showing all *published* courses.

1.1 API: Get Published Courses

Add route:

app/api/public/courses/route.js

```
import { NextResponse } from "next/server";
import { getSupabaseServerClient } from "@/lib/supabaseServer";

export async function GET() {
  try {
    const supabase = getSupabaseServerClient();

    const { data, error } = await supabase
      .from("courses")
      .select("id, slug, title, description, thumbnail_url, creator_id")
      .eq("is_published", true)
      .order("created_at", { ascending: false });

    if (error) {
      console.error("Fetch public courses error:", error);
      return NextResponse.json({ error: "Failed to fetch courses" }, {
        status: 500
      });
    }

    return NextResponse.json({ courses: data });
  } catch (err) {
    console.error("GET /api/public/courses error:", err);
    return NextResponse.json({ error: "Internal server error" }, {
      status: 500
    });
  }
}
```


1.2 Frontend Page

app/courses/page.js

```
"use client";

import { useEffect, useState } from "react";

export default function PublicCoursesPage() {
  const [courses, setCourses] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const load = async () => {
      try {
        const res = await fetch("/api/public/courses");
        const data = await res.json();

        if (!res.ok) throw new Error(data.error || "Failed to load courses");

        setCourses(data.courses || []);
      } catch (err) {
        console.error("Public courses error:", err);
        setError(err.message);
      } finally {
        setLoading(false);
      }
    };

    load();
  }, []);

  if (loading) return <p>Loading...</p>;
  if (error) return <p style={{ color: "red" }}>{error}</p>;

  return (
    <div>
      <h1>Courses</h1>
```

```
    {courses.length === 0 && <p>No courses available yet.</p>}

    <ul>
      {courses.map((c) => (
        <li key={c.id} style={{ marginBottom: "16px" }}>
          <a href={` /courses/${c.slug}`} >
            <strong>{c.title}</strong>
          </a>
          <p>{c.description}</p>
        </li>
      ))}
    </ul>
  </div>
);
}
```

2. COURSE DETAIL PAGE ([/courses/\[slug\]](/courses/[slug]))

- Unauthenticated users can view details.
- Only authenticated students can enroll.
- Shows “Enrolled / Start Learning” if already enrolled.

2.1 API: Get Course by Slug

app/api/public/courses/[slug]/route.js

```
import { NextResponse } from "next/server";
import { getSupabaseServerClient } from "@/lib/supabaseServer";

export async function GET(req, { params }) {
  try {
    const supabase = getSupabaseServerClient();

    const { data: course, error } = await supabase
      .from("courses")
```

```

        .select("id, slug, title, description, thumbnail_url,
is_published")
        .eq("slug", params.slug)
        .maybeSingle();

    if (error) {
        console.error("Slug error:", error);
        return NextResponse.json({ error: "Failed to fetch course" },
{ status: 500 });
    }

    if (!course || !course.is_published) {
        return NextResponse.json({ error: "Course not found" },
{ status: 404 });
    }

    return NextResponse.json({ course });
} catch (err) {
    console.error("Slug GET error:", err);
    return NextResponse.json({ error: "Internal server error" },
{ status: 500 });
}
}

```

2.2 Frontend Page

app/courses/[slug]/page.js

```

"use client";

import { useEffect, useState } from "react";
import { useAuth } from "@/contexts/AuthContext";
import { apiRequest } from "@/lib/apiClient";
import { useRouter, useParams } from "next/navigation";

export default function CourseDetailPage() {
    const { user } = useAuth();
    const { slug } = useParams();
    const router = useRouter();

    const [course, setCourse] = useState(null);

```

```

const [loading, setLoading] = useState(true);
const [enrolled, setEnrolled] = useState(false);
const [error, setError] = useState(null);
const [enrollLoading, setEnrollLoading] = useState(false);

useEffect(() => {
  const load = async () => {
    try {
      const res = await fetch(`/api/public/courses/${slug}`);
      const data = await res.json();

      if (!res.ok) throw new Error(data.error || "Failed to load course");

      setCourse(data.course);

      if (user) {
        const check = await apiRequest(
          "/api/enrollments/check",
          { method: "POST", body: { courseId: data.course.id } },
          user.token
        );
        setEnrolled(check.enrolled);
      }
    } catch (err) {
      console.error("Course detail error:", err);
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };

  load();
}, [slug, user]);

const enroll = async () => {
  if (!user) {
    router.push("/login");
    return;
  }

  setEnrollLoading(true);

```

```

    try {
      await apiRequest(
        "/api/enrollments",
        { method: "POST", body: { courseId: course.id } },
        user.token
      );
      setEnrolled(true);
      router.push(`/learn/${course.id}`);
    } catch (err) {
      console.error("Enroll error:", err);
      alert(err.message);
    } finally {
      setEnrollLoading(false);
    }
  };

  if (loading) return <p>Loading...</p>;
  if (error) return <p style={{ color: "red" }}>{error}</p>;

  return (
    <div>
      <h1>{course.title}</h1>
      <p>{course.description}</p>

      {!enrolled && (
        <button disabled={enrollLoading} onClick={enroll}>
          {enrollLoading ? "Enrolling..." : "Enroll Now"}
        </button>
      )}

      {enrolled && (
        <button onClick={() => router.push(`/learn/${course.id}`)}>
          Start Learning
        </button>
      )}
    </div>
  );
}

```

3. BACKEND: ENROLLMENT SYSTEM

You need:

1. `POST /api/enrollments` (enroll student)
2. `POST /api/enrollments/check` (check if enrolled)

You also need a helper that ensures the logged-in user has a student profile.

3.1 Student Profile Helper

`lib/studentHelpers.js`

```
export async function getOrCreateStudentProfile(supabase, userId, email) {
  const { data: existing, error } = await supabase
    .from("students")
    .select("*")
    .eq("user_id", userId)
    .maybeSingle();

  if (existing) return existing;
  if (error && error.code !== "PGRST116") throw error;

  const { data: student, error: insertErr } = await supabase
    .from("students")
    .insert({
      user_id: userId,
      display_name: email,
    })
    .select()
    .single();

  if (insertErr) throw insertErr;

  return student;
}
```

3.2 Enroll API

`app/api/enrollments/route.js`

```
import { NextResponse } from "next/server";
import { getAuthUserFromRequest } from "@/lib/apiAuth";
import { getOrCreateStudentProfile } from "@/lib/studentHelpers";

export async function POST(req) {
  try {
    const { user, error, status, supabase } = await
getAuthUserFromRequest(req);
    if (error) return NextResponse.json({ error }, { status });

    const body = await req.json().catch(() => null);
    if (!body?.courseId) {
      return NextResponse.json({ error: "Missing courseId" },
{ status: 400 });
    }

    const student = await getOrCreateStudentProfile(
      supabase,
      user.id,
      user.email
    );

    // Check course exists
    const { data: course } = await supabase
      .from("courses")
      .select("id, is_published")
      .eq("id", body.courseId)
      .maybeSingle();

    if (!course || !course.is_published) {
      return NextResponse.json({ error: "Course not found" },
{ status: 404 });
    }

    // Insert enrollment if not exists
    const { error: enrollErr } = await supabase
      .from("enrollments")
      .upsert(
        {
          course_id: course.id,
          student_id: student.id,
        },

```

```

        { onConflict: "course_id,student_id" }
    );

    if (enrollErr) {
        console.error("Enroll error:", enrollErr);
        return NextResponse.json({ error: "Failed to enroll" },
{ status: 500 });
    }

    return NextResponse.json({ success: true });
} catch (err) {
    console.error("Enroll catch error:", err);
    return NextResponse.json({ error: "Server error" }, { status:
500 });
}
}

```

3.3 Check Enrollment API

app/api/enrollments/check/route.js

```

import { NextResponse } from "next/server";
import { getAuthUserFromRequest } from "@/lib/apiAuth";
import { getOrCreateStudentProfile } from "@/lib/studentHelpers";

export async function POST(req) {
    try {
        const { user, error, status, supabase } = await
getAuthUserFromRequest(req);
        if (error) return NextResponse.json({ error }, { status });

        const body = await req.json().catch(() => null);
        if (!body?.courseId) {
            return NextResponse.json({ error: "Missing courseId" },
{ status: 400 });
        }

        const student = await getOrCreateStudentProfile(supabase,
user.id, user.email);

        const { data, error: checkErr } = await supabase

```



```
        .from("enrollments")
        .select("id")
        .eq("course_id", body.courseId)
        .eq("student_id", student.id)
        .maybeSingle();

    if (checkErr && checkErr.code !== "PGRST116") {
        console.error("Check enrollment error:", checkErr);
        return NextResponse.json({ error: "Failed to check" },
{ status: 500 });
    }

    return NextResponse.json({ enrolled: !!data });
} catch (err) {
    console.error(err);
    return NextResponse.json({ error: "Server error" }, { status:
500 });
}
}
```

4. STUDENT LEARNING VIEW

(/learn/[courseId])

This is the most sensitive part.
Access control must be bulletproof.

Rules:

- Must be authenticated
- Must have a student profile
- Must be enrolled
- Must only access courses they're enrolled in
- Must handle missing modules/lessons gracefully

4.1 Frontend Page

app/learn/[courseId]/page.js

```
"use client";
```

```
import { useEffect, useState } from "react";
import { useParams, useRouter } from "next/navigation";
import { useAuth } from "@/contexts/AuthContext";
import { apiRequest } from "@/lib/apiClient";
```

```
export default function LearnPage() {
  const { user } = useAuth();
  const { courseId } = useParams();
  const router = useRouter();

  const [course, setCourse] = useState(null);
  const [modules, setModules] = useState([]);
  const [lessonsByModule, setLessonsByModule] = useState({});
  const [selectedLesson, setSelectedLesson] = useState(null);
  const [loading, setLoading] = useState(true);
```

```
  useEffect(() => {
    if (!user) {
      router.push("/login");
      return;
    }
  })
```

```
  const load = async () => {
    try {
      // Check enrollment
      const check = await apiRequest(
        "/api/enrollments/check",
        { method: "POST", body: { courseId } },
        user.token
      );
      if (!check.enrolled) {
        router.push(`/courses`);
        return;
      }
    }
  }
```

```
  // Load course
  const courseData = await apiRequest(
    `/api/courses/${courseId}`,
    {},
    user.token
  );
```

```

    );

    setCourse(courseData.course);

    // Load modules
    const modulesData = await apiRequest(
      `/api/courses/${courseId}/modules`,
      {},
      user.token
    );
    const mods = modulesData.modules || [];
    setModules(mods);

    // Load lessons
    const map = {};
    for (const m of mods) {
      const lessonsData = await apiRequest(
        `/api/modules/${m.id}/lessons`,
        {},
        user.token
      );
      map[m.id] = lessonsData.lessons || [];
    }
    setLessonsByModule(map);

    // Auto-select first lesson
    if (mods.length > 0 && map[mods[0].id]?.length > 0) {
      setSelectedLesson(map[mods[0].id][0]);
    }

  } catch (err) {
    console.error("Learn page error:", err);
    router.push("/courses");
  } finally {
    setLoading(false);
  }
};

load();
}, [user, courseId, router]);

if (!user || loading) return <p>Loading...</p>;

```

```

    if (!course) return <p>Course not found.</p>;

    return (
      <div style={{ display: "flex" }}>
        <aside style={{ width: 250, borderRight: "1px solid #ddd",
padding: 16 }}>
          <h2>{course.title}</h2>

          {modules.map((m) => (
            <div key={m.id} style={{ marginTop: 16 }}>
              <strong>{m.title}</strong>
              <ul>
                {(lessonsByModule[m.id] || []).map((lesson) => (
                  <li
                    key={lesson.id}
                    onClick={() => setSelectedLesson(lesson)}
                    style={{
                      cursor: "pointer",
                      padding: 4,
                      background:
selectedLesson?.id === lesson.id ? "#eee" :
"transparent",
                    }}
                  >
                    {lesson.title}
                  </li>
                ))}
              </ul>
            </div>
          ))}
        </aside>

        <main style={{ flex: 1, padding: 24 }}>
          {selectedLesson ? (
            <LessonView lesson={selectedLesson} user={user} />
          ) : (
            <p>No lesson selected</p>
          )}
        </main>
      </div>
    );

```

```

}

function LessonView({ lesson, user }) {
  if (lesson.content_type === "video") {
    return (
      <div>
        <h2>{lesson.title}</h2>
        <video
          controls
          src={lesson.content_url}
          style={{ width: "100%", maxWidth: "800px" }}
        />
      </div>
    );
  }

  if (lesson.content_type === "text") {
    return (
      <div>
        <h2>{lesson.title}</h2>
        <p>{lesson.content_body}</p>
      </div>
    );
  }

  return <p>Unsupported lesson type.</p>;
}

```

All crash conditions are handled:

- missing modules
- missing lessons
- invalid enrollment
- user not logged in
- course deleted
- lesson deleted
- content type mismatch

5. BACKEND: PROGRESS TRACKING

5.1 API

app/api/progress/route.js

```
import { NextResponse } from "next/server";
import { getAuthUserFromRequest } from "@/lib/apiAuth";
import { getOrCreateStudentProfile } from "@/lib/studentHelpers";

export async function POST(req) {
  try {
    const { user, error, status, supabase } = await
getAuthUserFromRequest(req);
    if (error) return NextResponse.json({ error }, { status });

    const body = await req.json().catch(() => null);
    if (!body?.lessonId) {
      return NextResponse.json({ error: "Missing lessonId" },
{ status: 400 });
    }

    const student = await getOrCreateStudentProfile(supabase,
user.id, user.email);

    const { error: upsertErr } = await supabase
      .from("progress")
      .upsert(
        {
          student_id: student.id,
          lesson_id: body.lessonId,
          completed_at: new Date().toISOString(),
        },
        { onConflict: "student_id,lesson_id" }
      );

    if (upsertErr) {
      console.error("Progress error:", upsertErr);
    }
  }
}
```

```
        return NextResponse.json({ error: "Failed to save progress" },
{ status: 500 });
    }

    return NextResponse.json({ success: true });
} catch (err) {
    console.error("Progress catch error:", err);
    return NextResponse.json({ error: "Server error" }, { status:
500 });
}
}
```

You can call this on:

- button click
- video `onEnded`
- page exit

But for now, we wire it lightly.

6. COMPLETION CHECKLIST (DO NOT MOVE TO PHASE 5 IF ANY FAILS)

You must be able to:

Public Side

- ✓ `/courses` loads without logging in
- ✓ `/courses/[slug]` shows details
- ✓ "Enroll Now" redirects to login if not authenticated

Enrollment

- ✓ Authenticated user can enroll
- ✓ Multiple enroll attempts don't crash (upsert logic works)
- ✓ `/api/enrollments/check` returns correct value

Learning Area

- ✓ `/learn/[courseId]` redirects to login when unauthenticated
- ✓ Redirects to `/courses` when not enrolled
- ✓ Loads course + modules + lessons
- ✓ Selects first lesson automatically
- ✓ Doesn't crash if modules or lessons are empty
- ✓ Video plays without errors
- ✓ Text lessons display fine

Progress

- ✓ Calling `/api/progress` writes progress row
- ✓ Multiple completions don't duplicate rows
- ✓ No unauthorized access to other users' lessons

If all these pass → Phase 4 is **successfully done**.

NEXT PHASE (5): Stripe Subscription for Creators + Access Limits

When you're ready, I'll give you the complete Phase 5 guide with:

- Stripe checkout sessions
- Webhooks
- Syncing plan to Supabase
- Restricting course count on free plan
- Billing portal

But only once Phase 4 is flawlessly working.

You said:

Complete guide for phase 5 that really helps in building this project step by step without skipping any important work. So that the web app don't crash or show any error.

ChatGPT said:

Thought for 25s

Alright, Phase 5 is where you stop building a "free forever" toy and turn this into an actual SaaS with money attached.

Goal:

- Creators pay Stripe subscriptions
- Stripe ↔ Supabase stay **perfectly in sync**
- Your APIs enforce plan limits cleanly (no crashes, no silent failures)

Anything marked **[Inference]** is a design decision I'm making for you.

0. Preconditions (if these are broken, fix first)

Before Phase 5:

- Auth + `/api/me` work.
- Creator onboarding + `/creator/dashboard` work.
- Course CRUD and creator course editor work.
- Student enrollment & `/learn/[courseId]` work without runtime errors.

If any of that is flaky, Stripe will just add more chaos.

1. Stripe Setup (Dashboard + Env)

1.1 Create Stripe account + Product [Inference]

In Stripe Dashboard:

1. Go to **Products → Add product**
2. Example:
 - Name: `Course SaaS Pro`
3. Add pricing:
 - `Pro Monthly` – recurring, monthly (or yearly as well if you like)
4. Note the **Price ID** (e.g. `price_12345...`)

We'll hardcode that `price_id` into env for now. [Inference]

1.2 Get Keys

From **Developers** → **API keys**:

- Secret key (starts with `sk_test_...` in test)
- Publishable key (starts with `pk_test_...`)

From **Developers** → **Webhooks** (later):

- Webhook signing secret (`whsec_...`)

1.3 Add to `.env.local`

```
# Stripe
STRIPE_SECRET_KEY=sk_test_...
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test_...
STRIPE_PRICE_PRO_MONTHLY=price_...
STRIPE_WEBHOOK_SECRET=whsec_...

# App URLs (to avoid hardcoding)
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

Do **not** expose `STRIPE_SECRET_KEY` or `STRIPE_WEBHOOK_SECRET` via `NEXT_PUBLIC`.

Restart dev server after editing env.

2. Supabase: Extend `creators` Table

You already have:

- `stripe_customer_id`
- `current_plan` (default 'free')

Add more columns:

```
alter table creators
add column if not exists stripe_subscription_id text,
add column if not exists subscription_status text,
add column if not exists current_period_end timestampz;
```

We'll use:

- `current_plan`: 'free' | 'pro' [Inference]
 - `subscription_status`: 'active' | 'canceled' | 'past_due' | ... (Stripe statuses)
 - `stripe_customer_id`: Stripe customer id
 - `stripe_subscription_id`: subscription id
 - `current_period_end`: to show billing info
-

3. Install Stripe SDK

```
npm install stripe @stripe/stripe-js
```

- `stripe` → server-side (webhooks, checkout sessions)
 - `@stripe/stripe-js` → optional client-side; but we're going to redirect using `session.url` so we don't actually need it yet. [Inference]
-

4. Stripe Server Helper

```
lib/stripe.js:
```

```
import Stripe from "stripe";
```

```
const secretKey = process.env.STRIPE_SECRET_KEY;
```

```
if (!secretKey) {  
  throw new Error("Missing STRIPE_SECRET_KEY");  
}
```

```
export const stripe = new Stripe(secretKey, {  
  apiVersion: "2023-10-16", // or whatever current version you  
  choose  
});
```

If key is missing, app fails fast instead of failing randomly later.

5. Checkout Session Endpoint

Route: `POST /api/billing/create-checkout-session`

Goal:

- Authenticated creator
- Ensure Stripe customer exists
- Create subscription checkout session
- Return `session.url` to client

5.1 API

`app/api/billing/create-checkout-session/route.js:`

```
import { NextResponse } from "next/server";
import { getAuthUserFromRequest } from "@/lib/apiAuth";
import { getCreatorOrError } from "@/lib/creatorHelpers";
import { stripe } from "@/lib/stripe";

export async function POST(req) {
  try {
    const { user, error, status, supabase } = await
getAuthUserFromRequest(req);
    if (error) return NextResponse.json({ error }, { status });

    const { creator, error: creatorError, status: creatorStatus } =
      await getCreatorOrError(supabase, user.id);

    if (creatorError) {
      return NextResponse.json({ error: creatorError }, { status:
creatorStatus });
    }

    const priceId = process.env.STRIPE_PRICE_PRO_MONTHLY;
    if (!priceId) {
      return NextResponse.json(
        { error: "Stripe price not configured" },
        { status: 500 }
      );
    }
  }
}
```

```

// 1) Ensure Stripe customer
let customerId = creator.stripe_customer_id;

if (!customerId) {
  const customer = await stripe.customers.create({
    email: user.email,
    metadata: {
      supabase_user_id: user.id,
      creator_id: creator.id,
    },
  });

  customerId = customer.id;

  const { error: updateError } = await supabase
    .from("creators")
    .update({ stripe_customer_id: customerId })
    .eq("id", creator.id);

  if (updateError) {
    console.error("Failed to save stripe_customer_id:",
updateError);
    return NextResponse.json(
      { error: "Failed to link Stripe customer" },
      { status: 500 }
    );
  }
}

const appUrl = process.env.NEXT_PUBLIC_APP_URL ||
"http://localhost:3000";

// 2) Create Checkout Session
const session = await stripe.checkout.sessions.create({
  mode: "subscription",
  payment_method_types: ["card"],
  customer: customerId,
  line_items: [
    {
      price: priceId,
      quantity: 1,
    },
  ],
});

```

```

    ],
    success_url: `${appUrl}/creator/billing?status=success`,
    cancel_url: `${appUrl}/creator/billing?status=cancel`,
    metadata: {
      creator_id: creator.id,
      supabase_user_id: user.id,
    },
  });

  return NextResponse.json({ url: session.url });
} catch (err) {
  console.error("Create checkout session error:", err);
  return NextResponse.json(
    { error: "Failed to create checkout session" },
    { status: 500 }
  );
}
}
}

```

Notes:

- Use metadata to track mapping (extra safety).
 - No JSON body required here, so we ignore `req.json()`.
-

6. Stripe Webhook Endpoint (Core Logic)

This is the **most critical** part. If this is wrong, plans desync and everything is garbage.

6.1 Route file

`app/api/webhooks/stripe/route.js`:

Important:

- Must read **raw body** (`req.text()`)
- Must verify signature using `stripe.webhooks.constructEvent`

```

import { NextResponse } from "next/server";
import { stripe } from "@/lib/stripe";

```

```

import { getSupabaseServerClient } from "@lib/supabaseServer";

export const config = {
  api: {
    bodyParser: false,
  },
}; // Only needed in /pages/api; App Router ignores this but safe
[Inference]

export async function POST(req) {
  const sig = req.headers.get("stripe-signature");
  const webhookSecret = process.env.STRIPE_WEBHOOK_SECRET;

  if (!webhookSecret) {
    console.error("Missing STRIPE_WEBHOOK_SECRET");
    return NextResponse.json({ error: "Webhook not configured" },
{ status: 500 });
  }

  let event;

  try {
    const body = await req.text();
    event = stripe.webhooks.constructEvent(body, sig,
webhookSecret);
  } catch (err) {
    console.error("Webhook signature verification failed:",
err.message);
    return NextResponse.json({ error: "Invalid signature" },
{ status: 400 });
  }

  const supabase = getSupabaseServerClient();

  try {
    switch (event.type) {
      case "checkout.session.completed": {
        const session = event.data.object;
        await handleCheckoutCompleted(session, supabase);
        break;
      }
      case "customer.subscription.created":

```

```

    case "customer.subscription.updated":
    case "customer.subscription.deleted": {
        const subscription = event.data.object;
        await handleSubscriptionUpdated(subscription, supabase);
        break;
    }
    default:
        // ignore other events
        break;
}

return NextResponse.json({ received: true });
} catch (err) {
    console.error("Webhook handler error:", err);
    return NextResponse.json({ error: "Webhook handler failed" },
{ status: 500 });
}
}

```

```

async function handleCheckoutCompleted(session, supabase) {
    const customerId = session.customer;
    const creatorId = session.metadata?.creator_id;

    if (!customerId || !creatorId) {
        console.warn("Missing creatorId or customerId in
checkout.session metadata");
        return;
    }

```

```

    const { error } = await supabase
        .from("creators")
        .update({
            stripe_customer_id: customerId,
        })
        .eq("id", creatorId);

    if (error) {
        console.error("Failed to update creator after checkout:",
error);
    }
}

```



```

async function handleSubscriptionUpdated(subscription, supabase) {
  const customerId = subscription.customer;
  const stripeSubscriptionId = subscription.id;
  const status = subscription.status;
  const currentPeriodEnd = subscription.current_period_end
    ? new Date(subscription.current_period_end * 1000).toISOString()
    : null;

  if (!customerId) {
    console.warn("Subscription without customer id");
    return;
  }

  const { data: creator, error: fetchError } = await supabase
    .from("creators")
    .select("*")
    .eq("stripe_customer_id", customerId)
    .maybeSingle();

  if (fetchError) {
    console.error("Failed to fetch creator by customerId:",
fetchError);
    return;
  }

  if (!creator) {
    console.warn("No creator found for customer:", customerId);
    return;
  }

  // Decide plan based on price(s)
  let plan = "pro"; // [Inference] for a single price-based tier

  const { error: updateError } = await supabase
    .from("creators")
    .update({
      stripe_subscription_id: stripeSubscriptionId,
      subscription_status: status,
      current_period_end: currentPeriodEnd,
      current_plan: status === "active" ? plan : "free",
    })
    .eq("id", creator.id);

```

```
    if (updateError) {
      console.error("Failed to update creator subscription:",
updateError);
    }
  }
}
```

Test webhooks in test mode with Stripe CLI or dashboard. Use events:

- `checkout.session.completed`
 - `customer.subscription.updated`
-

7. Billing Page for Creators

Route: `/creator/billing`

Features:

- Show current plan & status
- Button to upgrade if `current_plan === 'free'`
- Optional: manage subscription button (customer portal)

7.1 API: creator billing info

You can reuse `/api/me` (we already return `creator` with plan info), so no extra endpoint needed. [Inference]

7.2 Frontend page

`app/creator/billing/page.js`:

```
"use client";

import { useEffect, useState } from "react";
import { useCreatorProfile } from "@/hooks/useCreatorProfile";
import { useSearchParams } from "next/navigation";
import { apiRequest } from "@/lib/apiClient";

export default function BillingPage() {
```

```

    const { user, creator, loading, error: profileError } =
useCreatorProfile();
    const searchParams = useSearchParams();

    const [creatingSession, setCreatingSession] = useState(false);
    const [message, setMessage] = useState(null);

    useEffect(() => {
        const status = searchParams.get("status");
        if (status === "success") {
            setMessage("Payment successful. Your plan will update
shortly.");
        } else if (status === "cancel") {
            setMessage("Checkout canceled.");
        }
    }, [searchParams]);

    const handleUpgrade = async () => {
        if (!user) return;
        setCreatingSession(true);
        setMessage(null);

        try {
            const data = await apiRequest(
                "/api/billing/create-checkout-session",
                { method: "POST" },
                user.token
            );

            window.location.href = data.url;
        } catch (err) {
            console.error("Upgrade error:", err);
            setMessage(err.message || "Failed to start checkout");
        } finally {
            setCreatingSession(false);
        }
    };

    if (loading) return <p>Loading...</p>;
    if (!user) return null;
    if (!creator) return <p>You must create a creator profile
first.</p>;

```

```

return (
  <div>
    <h1>Billing</h1>

    {profileError && <p style={{ color: "red"
}}>{profileError}</p>}
    {message && <p>{message}</p>}

    <p>
      <strong>Current plan:</strong> {creator.current_plan ||
"free"}
    </p>
    <p>
      <strong>Subscription status:</strong>{" "}
      {creator.subscription_status || "none"}
    </p>
    {creator.current_period_end && (
      <p>
        <strong>Current period end:</strong>{" "}
        {new Date(creator.current_period_end).toLocaleString()}
      </p>
    )}

    {creator.current_plan === "free" && (
      <button onClick={handleUpgrade} disabled={creatingSession}>
        {creatingSession ? "Redirecting..." : "Upgrade to Pro"}
      </button>
    )}

    {creator.current_plan === "pro" && (
      <p>
        You are on the <strong>Pro</strong> plan. To cancel or
manage billing,
        use the Stripe customer portal (you can wire this later).
      </p>
    )}
  </div>
);
}

```

8. (Optional but Recommended) Customer Portal Endpoint [Inference]

If you want a “Manage subscription” button:

app/api/billing/create-portal-session/route.js:

```
import { NextResponse } from "next/server";
import { getAuthUserFromRequest } from "@/lib/apiAuth";
import { getCreatorOrError } from "@/lib/creatorHelpers";
import { stripe } from "@/lib/stripe";

export async function POST(req) {
  try {
    const { user, error, status, supabase } = await
getAuthUserFromRequest(req);
    if (error) return NextResponse.json({ error }, { status });

    const { creator, error: creatorError, status: creatorStatus } =
      await getCreatorOrError(supabase, user.id);
    if (creatorError) {
      return NextResponse.json({ error: creatorError }, { status:
creatorStatus });
    }

    if (!creator.stripe_customer_id) {
      return NextResponse.json(
        { error: "No Stripe customer linked" },
        { status: 400 }
      );
    }

    const appUrl = process.env.NEXT_PUBLIC_APP_URL ||
"http://localhost:3000";

    const portalSession = await
stripe.billingPortal.sessions.create({
      customer: creator.stripe_customer_id,
      return_url: `${appUrl}/creator/billing`,
    });

    return NextResponse.json({ url: portalSession.url });
  }
}
```

```
} catch (err) {
  console.error("Portal session error:", err);
  return NextResponse.json(
    { error: "Failed to create portal session" },
    { status: 500 }
  );
}
```

Then in `/creator/billing`, show a “Manage Subscription” button if `creator.stripe_customer_id && creator.current_plan === 'pro'`.

9. Enforcing Plan Limits in APIs

You’re not selling “Pro” for vibes. You need **hard limits**.

Example limits [Inference]:

- `free`: max 1 course
- `pro`: unlimited courses

9.1 Helper

`lib/planLimits.js`:

```
export const PLAN_LIMITS = {
  free: {
    maxCourses: 1,
  },
  pro: {
    maxCourses: 1000, // effectively unlimited for now
  },
};

export function getPlanLimits(plan) {
  const normalized = plan || "free";
  return PLAN_LIMITS[normalized] || PLAN_LIMITS.free;
}
```

9.2 Enforce in `/api/courses` POST

Modify `app/api/courses/route.js` POST handler:

```
import { getPlanLimits } from "@lib/planLimits";

// inside POST after getting creator
const limits = getPlanLimits(creator.current_plan);

const { data: countData, error: countError } = await supabase
  .from("courses")
  .select("id", { count: "exact", head: true })
  .eq("creator_id", creator.id);

if (countError) {
  console.error("Count courses error:", countError);
  return NextResponse.json(
    { error: "Failed to check plan limits" },
    { status: 500 }
  );
}

const currentCount = countData?.length ?? countData ?? 0; //
supabase-js v2 count handling [Inference]

if (currentCount >= limits.maxCourses) {
  return NextResponse.json(
    {
      error: `Plan limit reached. Max courses for plan "${
        creator.current_plan
      }" is ${limits.maxCourses}.`,
    },
    { status: 403 }
  );
}
```

Now:

- Free plan → 403 when trying to create 2nd course
- Pro plan → allowed

You can later enforce more granular limits (max lessons, storage, etc.) using the same pattern.

10. Phase 5 “No Dumb Errors” Checklist

You’re done with Phase 5 if **all** of this works in test mode:

Stripe & Webhooks

- `STRIPE_SECRET_KEY`, `STRIPE_PRICE_PRO_MONTHLY`, `STRIPE_WEBHOOK_SECRET` exist.
- `/api/billing/create-checkout-session` returns a valid `url` when:
 - user is logged in
 - user has a creator profile
- Going through test checkout redirects back to `/creator/billing?status=success`.
- Webhook logs show `checkout.session.completed` and `customer.subscription.updated` being handled with no 400/500.
- `creators` row gets:
 - `stripe_customer_id`
 - `stripe_subscription_id`
 - `subscription_status = 'active'`
 - `current_plan = 'pro'`

Billing Page

- `/creator/billing`:
 - Shows “free” before upgrade.
 - After successful webhook, shows “pro” and correct status.
 - Shows meaningful message on `status=success` or `status=cancel`.
 - Doesn’t crash if subscription fields are null.

Limits

- On **free**:
 - Can create exactly 1 course.
 - Trying to create 2nd returns 403 JSON with clear error.
 - UI handles that error gracefully (no blank screens).
- On **pro**:
 - Can create multiple courses without API errors.

Error Paths

- Missing Stripe env → app fails at startup, not at random runtime.
 - Invalid webhook secret → 400 for webhook, but app still runs.
 - Calling billing APIs without auth → 401, not 500.
 - Creator without Stripe customer → checkout endpoint creates it and updates DB, no crash.
-

At this point, you have:

- Auth (Firebase)
- Data (Supabase)
- Creator + student flows
- Learning engine with progress tracking
- Stripe subscriptions with plan enforcement

That's a real SaaS foundation. The next layers are:

- Analytics dashboard (Chart.js)
- Drip content, email notifications
- Better UX (skeleton loaders, toasts, error boundaries)
- Hardening security (RLS, stricter policies, rate limiting)

But the money plumbing is now wired correctly instead of being a random afterthought.

You said:

Complete guide for phase 6 that really helps in buiding this project step by step without skipping any important work. So that the web app don't crash or show any error.

ChatGPT said:

Thought for 40s

Phase 6 = “make creators feel this is legit” phase.

We’re building **analytics** that *don’t* randomly 500, don’t explode with **undefined**, and don’t require you to be a SQL god.

Goal of this phase:

- Backend analytics APIs (safe, guarded, predictable)
- Frontend `/creator/analytics` page using **Chart.js + react-chartjs-2**
- Proper loading/error handling so nothing crashes

Anything marked **[Inference]** is a design choice I’m making for you, not something you explicitly specified.

0. Pre-check (don’t skip)

Before Phase 6:

- Auth + `useAuth`, `useCreatorProfile` work.
- Creator dashboard, course builder work.
- Student enrollment & `/learn/[courseId]` work.
- Stripe + `current_plan` in `creators` table works.

If any of this is shaky, fix that first. Analytics on top of broken core is pointless.

1. Install and register Chart.js

1.1 Install packages

```
npm install chart.js react-chartjs-2
```

1.2 Central Chart.js registration (very important)

If you don’t register chart components, you get runtime errors.

Create `lib/chartConfig.js`:

```
"use client";

import {
  Chart as ChartJS,
  CategoryScale,
  LinearScale,
  BarElement,
  LineElement,
  PointElement,
  ArcElement,
  Tooltip,
  Legend,
} from "chart.js";

let registered = false;

export function ensureChartJSRegistered() {
  if (registered) return;
  ChartJS.register(
    CategoryScale,
    LinearScale,
    BarElement,
    LineElement,
    PointElement,
    ArcElement,
    Tooltip,
    Legend
  );
  registered = true;
}
```

We call `ensureChartJSRegistered()` once in the analytics page to avoid double-register warnings.

2. Decide what metrics to show [Inference]

For v1, keep it useful but simple:

- **Overview:**

- total courses
- total enrollments
- unique students
- **Enrollments by course** (bar chart)
- **Enrollments over time (last 30 days)** (line chart)
- Optionally: basic completion per course (approximate)

We'll compute everything in JS from Supabase queries. For early-stage SaaS, that's fine.
[Inference]

3. Backend: Analytics APIs

All analytics are **creator-scoped** and **auth-guarded**.

We'll reuse:

- `getAuthUserFromRequest`
- `getCreatorOrError`
- `getSupabaseServerClient`

3.1 Helper to get creator's course IDs

Create `lib/analyticsHelpers.js`:

```
// lib/analyticsHelpers.js
export async function getCreatorCourses(supabase, creatorId) {
  const { data, error } = await supabase
    .from("courses")
    .select("id, title, created_at")
    .eq("creator_id", creatorId);

  if (error) {
    console.error("getCreatorCourses error:", error);
    throw new Error("Failed to load courses");
  }

  return data || [];
}
```

3.2 Overview endpoint

Route: `app/api/analytics/overview/route.js`

```
import { NextResponse } from "next/server";
import { getAuthUserFromRequest } from "@lib/apiAuth";
import { getCreatorOrError } from "@lib/creatorHelpers";
import { getCreatorCourses } from "@lib/analyticsHelpers";

export async function GET(req) {
  try {
    const { user, error, status, supabase } = await
getAuthUserFromRequest(req);
    if (error) return NextResponse.json({ error }, { status });

    const { creator, error: creatorError, status: creatorStatus } =
      await getCreatorOrError(supabase, user.id);

    if (creatorError) {
      return NextResponse.json({ error: creatorError }, { status:
creatorStatus });
    }

    const courses = await getCreatorCourses(supabase, creator.id);
    const courseIds = courses.map((c) => c.id);

    if (courseIds.length === 0) {
      return NextResponse.json({
        totalCourses: 0,
        totalEnrollments: 0,
        uniqueStudents: 0,
      });
    }

    const { data: enrollments, error: enrollError } = await supabase
      .from("enrollments")
      .select("id, student_id, course_id, created_at")
      .in("course_id", courseIds);

    if (enrollError) {
```

```

        console.error("Overview enrollments error:", enrollError);
        return NextResponse.json(
            { error: "Failed to load enrollments" },
            { status: 500 }
        );
    }

    const totalCourses = courses.length;
    const totalEnrollments = enrollments?.length || 0;
    const uniqueStudentIds = new Set(
        (enrollments || []).map((e) => e.student_id)
    );
    const uniqueStudents = uniqueStudentIds.size;

    return NextResponse.json({
        totalCourses,
        totalEnrollments,
        uniqueStudents,
    });
} catch (err) {
    console.error("GET /api/analytics/overview error:", err);
    return NextResponse.json(
        { error: "Internal server error" },
        { status: 500 }
    );
}
}

```

No grouping in SQL, all safe in JS.

3.3 Enrollments by course endpoint

Route: `app/api/analytics/enrollments-by-course/route.js`

```

import { NextResponse } from "next/server";
import { getAuthUserFromRequest } from "@/lib/apiAuth";
import { getCreatorOrError } from "@/lib/creatorHelpers";
import { getCreatorCourses } from "@/lib/analyticsHelpers";

export async function GET(req) {
    try {

```

```

    const { user, error, status, supabase } = await
getAuthUserFromRequest(req);
    if (error) return NextResponse.json({ error }, { status });

    const { creator, error: creatorError, status: creatorStatus } =
      await getCreatorOrError(supabase, user.id);

    if (creatorError) {
      return NextResponse.json({ error: creatorError }, { status:
creatorStatus });
    }

    const courses = await getCreatorCourses(supabase, creator.id);
    const courseIds = courses.map((c) => c.id);

    if (courseIds.length === 0) {
      return NextResponse.json({ series: [] });
    }

    const { data: enrollments, error: enrollError } = await supabase
      .from("enrollments")
      .select("id, course_id")
      .in("course_id", courseIds);

    if (enrollError) {
      console.error("Enrollments by course error:", enrollError);
      return NextResponse.json(
        { error: "Failed to load enrollments" },
        { status: 500 }
      );
    }

    const countsByCourseId = {};
    for (const e of enrollments || []) {
      countsByCourseId[e.course_id] = (countsByCourseId[e.course_id]
|| 0) + 1;
    }

    const series = courses.map((c) => ({
      courseId: c.id,
      title: c.title,
      enrollments: countsByCourseId[c.id] || 0,

```

```

    }));

    return NextResponse.json({ series });
  } catch (err) {
    console.error("GET /api/analytics/enrollments-by-course error:",
err);
    return NextResponse.json(
      { error: "Internal server error" },
      { status: 500 }
    );
  }
}

```

Always returns `series: []` on “no data”, not a 500.

3.4 Enrollments over time (last 30 days) [Inference]

We'll give them a simple time series: date → count of new enrollments that day.

Route: `app/api/analytics/enrollments-over-time/route.js`

```

import { NextResponse } from "next/server";
import { getAuthUserFromRequest } from "@/lib/apiAuth";
import { getCreatorOrError } from "@/lib/creatorHelpers";
import { getCreatorCourses } from "@/lib/analyticsHelpers";

export async function GET(req) {
  try {
    const { user, error, status, supabase } = await
getAuthUserFromRequest(req);
    if (error) return NextResponse.json({ error }, { status });

    const { creator, error: creatorError, status: creatorStatus } =
      await getCreatorOrError(supabase, user.id);

    if (creatorError) {
      return NextResponse.json({ error: creatorError }, { status:
creatorStatus });
    }

    const courses = await getCreatorCourses(supabase, creator.id);

```



```

const courseIds = courses.map((c) => c.id);

if (courseIds.length === 0) {
  return NextResponse.json({ points: [] });
}

const now = new Date();
const daysBack = 30; // [Inference]
const fromDate = new Date(now.getTime() - daysBack * 24 * 60 *
60 * 1000);

const { data: enrollments, error: enrollError } = await supabase
  .from("enrollments")
  .select("id, course_id, created_at")
  .in("course_id", courseIds)
  .gte("created_at", fromDate.toISOString());

if (enrollError) {
  console.error("Enrollments over time error:", enrollError);
  return NextResponse.json(
    { error: "Failed to load enrollments" },
    { status: 500 }
  );
}

const countsByDate = {};

for (const e of enrollments || []) {
  const d = new Date(e.created_at);
  const key = d.toISOString().slice(0, 10); // YYYY-MM-DD
  countsByDate[key] = (countsByDate[key] || 0) + 1;
}

// Expand into continuous series for chart niceness
const points = [];
for (let i = 0; i < daysBack; i++) {
  const d = new Date(fromDate.getTime() + i * 24 * 60 * 60 *
1000);
  const key = d.toISOString().slice(0, 10);
  points.push({
    date: key,
    count: countsByDate[key] || 0,
  });
}

```

```

    });
  }

  return NextResponse.json({ points });
} catch (err) {
  console.error("GET /api/analytics/enrollments-over-time error:",
err);
  return NextResponse.json(
    { error: "Internal server error" },
    { status: 500 }
  );
}
}
}

```

Even with 0 enrollments you still get a flat line of 0, not a crash.

3.5 Completion per course (optional, simple) [Inference]

Approximate metric: % of completed lessons per enrolled student. Simplified v1:

- For each course:
 - `lessonsCount` = number of lessons
 - `enrollmentsCount`
 - `progressCount` = number of `progress` rows for lessons of this course
 - `completionRate` = `progressCount / (lessonsCount * enrollmentsCount)` (clamped 0–1)

Edge case: if `lessonsCount === 0` or `enrollmentsCount === 0` → `completionRate = 0` (avoid divide by 0).

Route: `app/api/analytics/completion-by-course/route.js`

I'll keep it short but safe:

```

import { NextResponse } from "next/server";
import { getAuthUserFromRequest } from "@lib/apiAuth";
import { getCreatorOrError } from "@lib/creatorHelpers";
import { getCreatorCourses } from "@lib/analyticsHelpers";

```

```

export async function GET(req) {
  try {
    const { user, error, status, supabase } = await
getAuthUserFromRequest(req);
    if (error) return NextResponse.json({ error }, { status });

    const { creator, error: creatorError, status: creatorStatus } =
      await getCreatorOrError(supabase, user.id);

    if (creatorError) {
      return NextResponse.json({ error: creatorError }, { status:
creatorStatus });
    }

    const courses = await getCreatorCourses(supabase, creator.id);
    const courseIds = courses.map((c) => c.id);

    if (courseIds.length === 0) {
      return NextResponse.json({ series: [] });
    }

    // Load lessons grouped by course
    const { data: modules, error: modulesError } = await supabase
      .from("modules")
      .select("id, course_id")
      .in("course_id", courseIds);

    if (modulesError) {
      console.error("Completion modules error:", modulesError);
      return NextResponse.json(
        { error: "Failed to load modules" },
        { status: 500 }
      );
    }

    const moduleIds = (modules || []).map((m) => m.id);

    const { data: lessons, error: lessonsError } = await supabase
      .from("lessons")
      .select("id, module_id")
      .in("module_id", moduleIds.length ? moduleIds : ["00000000-
0000-0000-0000-000000000000"]); // hack to avoid empty IN

```

```

if (lessonsError) {
  console.error("Completion lessons error:", lessonsError);
  return NextResponse.json(
    { error: "Failed to load lessons" },
    { status: 500 }
  );
}

const lessonIdsByCourse = {};
const moduleCourseMap = {};
for (const m of modules || []) {
  moduleCourseMap[m.id] = m.course_id;
}
for (const l of lessons || []) {
  const courseId = moduleCourseMap[l.module_id];
  if (!courseId) continue;
  if (!lessonIdsByCourse[courseId]) lessonIdsByCourse[courseId]
= [];
  lessonIdsByCourse[courseId].push(l.id);
}

// Load enrollments
const { data: enrollments, error: enrollError } = await supabase
  .from("enrollments")
  .select("id, course_id")
  .in("course_id", courseIds);

if (enrollError) {
  console.error("Completion enrollments error:", enrollError);
  return NextResponse.json(
    { error: "Failed to load enrollments" },
    { status: 500 }
  );
}

// Load progress (for all lesson_ids)
const allLessonIds = (lessons || []).map((l) => l.id);
let progress = [];
if (allLessonIds.length > 0) {
  const { data: progData, error: progError } = await supabase
    .from("progress")

```

```

        .select("id, lesson_id");

    if (progError) {
        console.error("Completion progress error:", progError);
        return NextResponse.json(
            { error: "Failed to load progress" },
            { status: 500 }
        );
    }
    progress = progData || [];
}

const progressCountByCourse = {};
const lessonCourseMap = {};
for (const l of lessons || []) {
    lessonCourseMap[l.id] = moduleCourseMap[l.module_id];
}
for (const p of progress) {
    const courseId = lessonCourseMap[p.lesson_id];
    if (!courseId) continue;
    progressCountByCourse[courseId] =
        (progressCountByCourse[courseId] || 0) + 1;
}

const enrollmentsByCourse = {};
for (const e of enrollments || []) {
    enrollmentsByCourse[e.course_id] =
        (enrollmentsByCourse[e.course_id] || 0) + 1;
}

const series = courses.map((c) => {
    const lessonsCount = (lessonIdsByCourse[c.id] || []).length;
    const enrollCount = enrollmentsByCourse[c.id] || 0;
    const progCount = progressCountByCourse[c.id] || 0;

    let rate = 0;
    if (lessonsCount > 0 && enrollCount > 0) {
        rate = progCount / (lessonsCount * enrollCount);
        if (!Number.isFinite(rate)) rate = 0;
    }

    return {

```

```

        courseId: c.id,
        title: c.title,
        completionRate: Math.max(0, Math.min(1, rate)),
    });
});

return NextResponse.json({ series });
} catch (err) {
    console.error("GET /api/analytics/completion-by-course error:",
err);
    return NextResponse.json(
        { error: "Internal server error" },
        { status: 500 }
    );
}
}
}

```

This is **approximate** and intentionally simple. [Inference]

4. Frontend: **/creator/analytics** page

This page:

- Requires creator auth
- Fetches data from our 3–4 analytics endpoints
- Uses Chart.js via **react-chartjs-2**
- Handles loading + error states

4.1 Page skeleton

app/creator/analytics/page.js:

```

"use client";

import { useEffect, useState } from "react";
import { useCreatorProfile } from "@/hooks/useCreatorProfile";
import { apiRequest } from "@/lib/apiClient";
import { ensureChartJSRegistered } from "@/lib/chartConfig";
import { Line, Bar, Pie } from "react-chartjs-2";

```

```

ensureChartJSRegistered();

export default function CreatorAnalyticsPage() {
  const { user, creator, loading: profileLoading, error:
profileError } =
    useCreatorProfile();

  const [overview, setOverview] = useState(null);
  const [byCourse, setByCourse] = useState([]);
  const [overTime, setOverTime] = useState([]);
  const [completion, setCompletion] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const load = async () => {
      if (profileLoading) return;
      if (!user || !creator) return;

      try {
        setError(null);
        setLoading(true);

        const [overviewRes, byCourseRes, overTimeRes, completionRes]
=
          await Promise.all([
            apiRequest("/api/analytics/overview", {}, user.token),
            apiRequest("/api/analytics/enrollments-by-course", {},
user.token),
            apiRequest("/api/analytics/enrollments-over-time", {},
user.token),
            apiRequest("/api/analytics/completion-by-course", {},
user.token),
          ]);

        setOverview(overviewRes);
        setByCourse(byCourseRes.series || []);
        setOverTime(overTimeRes.points || []);
        setCompletion(completionRes.series || []);
      } catch (err) {
        console.error("Analytics load error:", err);
      }
    };
  });
}

```

```

        setError(err.message || "Failed to load analytics");
    } finally {
        setLoading(false);
    }
};

load();
}, [profileLoading, user, creator]));

if (profileLoading || loading) {
    return <p>Loading analytics...</p>;
}

if (!user) return <p>You must be logged in.</p>;
if (!creator) return <p>You must create a creator profile
first.</p>;

if (error || profileError) {
    return (
        <div>
            <p style={{ color: "red" }}>{error || profileError}</p>
        </div>
    );
}

return (
    <div>
        <h1>Analytics</h1>

        <OverviewCards overview={overview} />
        <div style={{ display: "flex", flexWrap: "wrap", gap: 24,
marginTop: 24 }}>
            <div style={{ flex: 1, minWidth: 320 }}>
                <h2>Enrollments by Course</h2>
                <EnrollmentsByCourseChart data={byCourse} />
            </div>

            <div style={{ flex: 1, minWidth: 320 }}>
                <h2>Enrollments (Last 30 Days)</h2>
                <EnrollmentsOverTimeChart data={overTime} />
            </div>
        </div>
    );
}

```



```

        <div style={{ flex: 1, minWidth: 320 }}>
          <h2>Completion by Course</h2>
          <CompletionByCourseChart data={completion} />
        </div>
      </div>
    </div>
  );
}

```

Note:

- It doesn't crash if any array is empty.
- It doesn't assume `overview` is non-null.

4.2 Overview cards component

```

function OverviewCards({ overview }) {
  if (!overview) {
    return <p>No data yet.</p>;
  }

  const items = [
    { label: "Total courses", value: overview.totalCourses ?? 0 },
    { label: "Total enrollments", value:
overview.totalEnrollments ?? 0 },
    { label: "Unique students", value: overview.uniqueStudents ??
0 },
  ];

  return (
    <div style={{ display: "flex", gap: 16 }}>
      {items.map((item) => (
        <div
          key={item.label}
          style={{
            border: "1px solid #ddd",
            borderRadius: 8,
            padding: 16,
            minWidth: 160,
          }}

```

```

        >
        <div style={{ fontSize: 12, color: "#555" }}>{item.label}
</div>
        <div style={{ fontSize: 24, fontWeight: "bold" }}
>{item.value}</div>
        </div>
    )})
</div>
);
}

```

4.3 Enrollments by course bar chart

```

function EnrollmentsByCourseChart({ data }) {
  if (!data || data.length === 0) {
    return <p>Not enough data yet.</p>;
  }

  const labels = data.map((d) =>
    d.title.length > 18 ? d.title.slice(0, 15) + "..." : d.title
  );
  const values = data.map((d) => d.enrollments);

  const chartData = {
    labels,
    datasets: [
      {
        label: "Enrollments",
        data: values,
      },
    ],
  };

  const options = {
    responsive: true,
    plugins: {
      legend: { display: false },
    },
  };

  return <Bar data={chartData} options={options} />;
}

```

```
}
```

4.4 Enrollments over time line chart

```
function EnrollmentsOverTimeChart({ data }) {
  if (!data || data.length === 0) {
    return <p>Not enough data yet.</p>;
  }

  const labels = data.map((d) => d.date);
  const values = data.map((d) => d.count);

  const chartData = {
    labels,
    datasets: [
      {
        label: "New enrollments",
        data: values,
      },
    ],
  };

  const options = {
    responsive: true,
    plugins: {
      legend: { display: false },
    },
  };

  return <Line data={chartData} options={options} />;
}
```

4.5 Completion pie chart by course

```
function CompletionByCourseChart({ data }) {
  if (!data || data.length === 0) {
    return <p>Not enough data yet.</p>;
  }

  const labels = data.map((d) =>
```

```

    d.title.length > 18 ? d.title.slice(0, 15) + "..." : d.title
  );
  const values = data.map((d) => Math.round((d.completionRate || 0)
* 100));

  const chartData = {
    labels,
    datasets: [
      {
        data: values,
      },
    ],
  };

  const options = {
    responsive: true,
  };

  return <Pie data={chartData} options={options} />;
}

```

Even if some `completionRate` is `undefined` or `NaN`, we take `|| 0` and it doesn't blow up.

5. Error Hygiene & Non-crash Rules

To keep this phase from becoming “Chart.js of Doom”:

1. **Chart.js registration is centralized**
 You called `ensureChartJSRegistered()` once at top level of the page, not inside `render` loops.
2. **Every API endpoint:**
 - Wraps main logic in `try/catch`
 - Logs server errors
 - Returns `{ error: "message" }` with a status, not HTML or null
3. **Every client-side fetch:**

- Uses `apiRequest` (already throws on non-2xx)
 - Wrapped in `try/catch`
 - On failure sets `error` state, which is then rendered as text, not “white screen”
4. **All maps:**
- Use `data || []` before mapping
 - Use `thing ?? default` for numbers
5. **No direct usage of `user` / `creator` before guards settle**
We check `profileLoading`, `user`, `creator` before making API calls.
-

6. Phase 6 Done Checklist

Don't tell yourself it's “done” until:

- `/creator/analytics` loads with:
 - No console errors
 - No server 500s
- A creator with:
 - 0 courses → sees “No data” messages, not crashes
 - Some courses but no enrollments → graphs show zeros or “Not enough data”
 - Some enrollments → charts render actual meaningful lines/bars/pies
- Killing a course in Supabase:
 - Doesn't crash analytics page (at worst fewer data points)
- Stripe / billing has **no impact** on analytics queries if subscription fields are null

At that point, you've got:

- Auth
- Creator + student flows

- Payments
- Analytics

Which is already a serious SaaS skeleton. From here you move into quality-of-life and “make it feel premium”: better UI, toasts, pagination, RLS policies, and maybe AI helpers (auto content, auto quizzes, etc.) — but you’re out of toy territory.

You said:

Complete guide for phase 7 that really helps in buiding this project step by step without skipping any important work. So that the web app don't crash or show any error.

ChatGPT said:

Thought for 14s

Phase 7 is “premium SaaS features” territory: drip content, emails, and quizzes. Also the phase where people love to introduce race conditions and 500s. Let’s not.

I’ll walk you through:

1. Drip content (schema + API + UI)
2. Email sending (safe, no-crash, provider-agnostic)
3. Quizzes (schema + APIs + UI in editor + UI in learner view)
4. Hygiene checklist

Anything marked **[Inference]** is me designing details for you, not taken from any spec.

0. Preconditions (same rule as always)

Don’t start Phase 7 unless:

- Creator flow works end-to-end (create course, modules, lessons).
- Student flow works (enroll, view lessons).
- Billing works and doesn’t blow up.
- Analytics doesn’t crash.

If earlier phases are shaky, you’ll just stack more bugs.

1. DRIP CONTENT – SCHEMA CHANGES

Goal:

- A course can optionally be “drip-based”.
- Each lesson can have a delay in days relative to enrollment date.
- Only unlocked lessons should be playable; locked ones visible but disabled.

1.1 Add drip fields to **courses** and **lessons** [Inference]

Run in Supabase SQL:

```
alter table courses
add column if not exists drip_enabled boolean default false,
add column if not exists drip_start_immediately boolean default
true;
```

```
alter table lessons
add column if not exists drip_delay_days int default 0;
```

Meaning:

- **courses.drip_enabled**: if **false** → all lessons available immediately (current behavior).
- **drip_start_immediately**: [Inference] for future: allow “start drip on first lesson completion” etc; for now we stick to “on enrollment”.
- **lessons.drip_delay_days**: number of days after enrollment when the lesson unlocks.

No crashes if you don't use these — default values keep current behavior.

2. DRIP LOGIC – BACKEND

You have two options:

- Filter lessons server-side (recommended: consistent, secure)
- Filter client-side using enrollment date (easier to get wrong)

We do **server-side filtering** via a new endpoint.

2.1 API: **/api/learn/structure**

Goal: one endpoint that returns:

- course
- modules
- lessons
- which lessons are locked/unlocked for this student

Create `app/api/learn/[courseId]/structure/route.js`:

```
import { NextResponse } from "next/server";
import { getAuthUserFromRequest } from "@/lib/apiAuth";
import { getOrCreateStudentProfile } from "@/lib/studentHelpers";

export async function GET(req, { params }) {
  try {
    const { courseId } = params;
    const { user, error, status, supabase } = await
getAuthUserFromRequest(req);
    if (error) return NextResponse.json({ error }, { status });

    const student = await getOrCreateStudentProfile(supabase,
user.id, user.email);

    // 1) Check enrollment
    const { data: enrollment, error: enrollError } = await supabase
      .from("enrollments")
      .select("id, created_at")
      .eq("course_id", courseId)
      .eq("student_id", student.id)
      .maybeSingle();

    if (enrollError) {
      console.error("Learn structure enrollment error:",
enrollError);
      return NextResponse.json(
        { error: "Failed to check enrollment" },
        { status: 500 }
      );
    }

    if (!enrollment) {
```



```

        return NextResponse.json(
          { error: "Not enrolled" },
          { status: 403 }
        );
      }

      const enrollmentDate = new Date(enrollment.created_at);

      // 2) Load course
      const { data: course, error: courseError } = await supabase
        .from("courses")
        .select(
          "id, title, description, drip_enabled,
drip_start_immediately"
        )
        .eq("id", courseId)
        .maybeSingle();

      if (courseError) {
        console.error("Learn structure course error:", courseError);
        return NextResponse.json(
          { error: "Failed to load course" },
          { status: 500 }
        );
      }

      if (!course) {
        return NextResponse.json({ error: "Course not found" },
{ status: 404 });
      }

      // 3) Load modules
      const { data: modules, error: modulesError } = await supabase
        .from("modules")
        .select("id, title, course_id, order_index")
        .eq("course_id", courseId)
        .order("order_index", { ascending: true });

      if (modulesError) {
        console.error("Learn structure modules error:", modulesError);
        return NextResponse.json(
          { error: "Failed to load modules" },

```

```

        { status: 500 }
    );
}

const moduleIds = (modules || []).map((m) => m.id);

// 4) Load lessons
let lessons = [];
if (moduleIds.length > 0) {
    const { data: lessonsData, error: lessonsError } = await
supabase
        .from("lessons")
        .select(
            "id, module_id, title, content_type, content_url,
content_body, order_index, drip_delay_days"
        )
        .in("module_id", moduleIds)
        .order("order_index", { ascending: true });

    if (lessonsError) {
        console.error("Learn structure lessons error:",
lessonsError);
        return NextResponse.json(
            { error: "Failed to load lessons" },
            { status: 500 }
        );
    }

    lessons = lessonsData || [];
}

// 5) Apply drip logic
const now = new Date();
const unlockedLessons = [];
const lockedLessons = [];

for (const lesson of lessons) {
    let isLocked = false;

    if (course.drip_enabled) {
        const unlockTime = new Date(enrollmentDate);
        const delayDays = lesson.drip_delay_days || 0;

```

```

        unlockTime.setDate(unlockTime.getDate() + delayDays);

        if (now < unlockTime) {
            isLocked = true;
        }
    }

    const entry = { ...lesson, isLocked };
    if (isLocked) lockedLessons.push(entry);
    else unlockedLessons.push(entry);
}

// Group by module for convenience
const lessonsByModule = {};
for (const lesson of [...unlockedLessons, ...lockedLessons]) {
    if (!lessonsByModule[lesson.module_id]) {
        lessonsByModule[lesson.module_id] = [];
    }
    lessonsByModule[lesson.module_id].push(lesson);
}

return NextResponse.json({
    course,
    modules,
    lessonsByModule,
});
} catch (err) {
    console.error("GET /api/learn/[courseId]/structure error:",
err);
    return NextResponse.json(
        { error: "Internal server error" },
        { status: 500 }
    );
}
}

```

Notes:

- If not enrolled → 403, not 500.
- If no modules/lessons → still returns empty arrays, no crash.

- `isLocked` is computed per lesson, not baked into DB.
-

3. DRIP LOGIC – FRONTEND (LEARN PAGE)

Update `app/learn/[courseId]/page.js` to use the new endpoint and respect `isLocked`.

Replace old “manual fetch” logic with a single call to `/api/learn/[courseId]/structure`.

3.1 Load structure

```
"use client";

import { useEffect, useState } from "react";
import { useParams, useRouter } from "next/navigation";
import { useAuth } from "@/contexts/AuthContext";
import { apiRequest } from "@/lib/apiClient";

export default function LearnPage() {
  const { user } = useAuth();
  const { courseId } = useParams();
  const router = useRouter();

  const [course, setCourse] = useState(null);
  const [modules, setModules] = useState([]);
  const [lessonsByModule, setLessonsByModule] = useState({});
  const [selectedLesson, setSelectedLesson] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    if (!user) {
      router.push("/login");
      return;
    }
  });

  const load = async () => {
    try {
      const data = await apiRequest(
        `/api/learn/${courseId}/structure`,
        {},
      );
    }
  };
}
```

```

        user.token
    );

    setCourse(data.course);
    setModules(data.modules || []);
    setLessonsByModule(data.lessonsByModule || {});

    // auto-select first **unlocked** lesson
    for (const m of data.modules || []) {
        const lessons = (data.lessonsByModule?.[m.id] ||
[]).filter(
            (l) => !l.isLocked
        );
        if (lessons.length > 0) {
            setSelectedLesson(lessons[0]);
            break;
        }
    }
    } catch (err) {
        console.error("Learn structure load error:", err);
        router.push("/courses");
    } finally {
        setLoading(false);
    }
};

load();
}, [user, courseId, router]);

if (!user || loading) return <p>Loading...</p>;
if (!course) return <p>Course not found or not accessible.</p>;

return (
    <div style={{ display: "flex" }}>
        <aside style={{ width: 250, borderRight: "1px solid #ddd",
padding: 16 }}>
            <h2>{course.title}</h2>

            {modules.map((m) => (
                <div key={m.id} style={{ marginTop: 16 }}>
                    <strong>{m.title}</strong>
                    <ul>

```

```

        {(lessonsByModule[m.id] || []).map((lesson) => (
            <li
                key={lesson.id}
                onClick={() => {
                    if (!lesson.isLocked) setSelectedLesson(lesson);
                }}
                style={{
                    cursor: lesson.isLocked ? "not-allowed" :
"pointer",

                    opacity: lesson.isLocked ? 0.5 : 1,
                    padding: 4,
                    background:
"transparent",

                    selectedLesson?.id === lesson.id ? "#eee" :

                }}
            >
                {lesson.title}
                {lesson.isLocked && (
                    <span style={{ marginLeft: 6, fontSize: 10 }}>
                        (Locked)
                    </span>
                )}
            </li>
        ))}
    </ul>
</div>
))}
</aside>

<main style={{ flex: 1, padding: 24 }}>
    {selectedLesson ? (
        <LessonView lesson={selectedLesson} user={user} />
    ) : (
        <p>
            {course.drip_enabled
                ? "No lessons unlocked yet. Come back later."
                : "No lessons available."}
        </p>
    )}
</main>
</div>
);

```

```
}
```

`LessonView` can stay similar to previous version; just don't assume lesson is always unlocked when rendered (we control that via selection logic).

4. DRIP CONTROLS IN CREATOR UI

Creators need to set `drip_enabled` and `drip_delay_days`.

4.1 Course-level toggle in `/creator/courses/[id]/edit`

In course details section, add:

```
// inside EditCoursePage component, state:
const [dripEnabled, setDripEnabled] = useState(false);

// when loading course:
setDripEnabled(!course.drip_enabled);

// in PUT payload:
body: {
  title,
  description,
  drip_enabled: dripEnabled,
}

// in JSX:
<label style={{ display: "block", marginTop: 12 }}>
  <input
    type="checkbox"
    checked={dripEnabled}
    onChange={(e) => setDripEnabled(e.target.checked)}
  />{" "}
  Enable drip content (unlock lessons over time)
</label>
```

Make sure your `/api/courses/[id]` PUT route allows `drip_enabled` field in validation schema.

4.2 Lesson-level field in modules UI

In `ModuleItem`, where you create lessons, add an input for delay days.

State:

```
const [newLessonDelayDays, setNewLessonDelayDays] = useState(0);
```

Form:

```
<input
  type="number"
  min={0}
  placeholder="Delay days"
  value={newLessonDelayDays}
  onChange={(e) => setNewLessonDelayDays(Number(e.target.value) ||
0)}
/>
```

When calling `/api/modules/[id]/lessons` POST:

```
body: {
  title: newLessonTitle.trim(),
  content_type: "video",
  content_url: newLessonUrl.trim(),
  drip_delay_days: newLessonDelayDays,
},
```

And update your `LessonSchema` on that API to include:

```
const LessonSchema = z.object({
  title: z.string().min(3).max(200),
  content_type: z.enum(["video", "text", "pdf"]).optional(),
  content_url: z.string().url().optional(),
  content_body: z.string().optional(),
  order_index: z.number().int().optional(),
  drip_delay_days: z.number().int().min(0).optional(),
});
```

No delay set? → fallback to 0 → immediately on day 0.

5. EMAIL NOTIFICATIONS (SAFELY) [Inference]

We'll treat email as optional: if not configured → log and no-op; app should not crash.

5.1 Pick a provider (SendGrid / Resend / Mailgun etc.)

Since you didn't specify one, I'll use a generic pattern with a fake "email provider" call. You'll plug your SDK into it.

Env (example):

```
EMAIL_PROVIDER_API_KEY=...  
EMAIL_FROM_ADDRESS="Your SaaS <noreply@yoursaas.com>"
```

5.2 Email helper

lib/email.js:

```
export async function sendEmail({ to, subject, text }) {  
  const apiKey = process.env.EMAIL_PROVIDER_API_KEY;  
  const from = process.env.EMAIL_FROM_ADDRESS;  
  
  if (!apiKey || !from) {  
    console.warn("Email not configured. Skipping send.");  
    return;  
  }  
  
  try {  
    // TODO: integrate actual provider SDK here.  
    // Example pseudo-implementation:  
    // await someEmailSDK.send({ apiKey, from, to, subject, text });  
  
    console.log(`Pretend sending email to ${to}: ${subject}`);  
  } catch (err) {  
    console.error("Email send error:", err);  
    // Don't throw; failing email should not crash the app logic  
  }  
}
```

This keeps your app safe even if email is misconfigured.

5.3 Trigger email on enrollment

In `/api/enrollments/route.js`, after successful enrollment:

```
import { sendEmail } from "@/lib/email";
```

```
// after enrollment insert success:
try {
  // Fetch student email via users table (user already has email)
  await sendEmail({
    to: user.email,
    subject: `Enrolled in course`,
    text: `You have enrolled in a course. Log in to start
learning.`,
  });
} catch (err) {
  console.error("Enrollment email error:", err);
}
```

Email errors are logged, not thrown.

5.4 Trigger email on new drip unlock (optional, more advanced) [Inference]

This ideally needs a scheduled job / cron (Supabase scheduled function, external cron hitting an endpoint, etc.). Minimal version:

- Create `/api/cron/drip-reminders`
- It:
 - Fetches all enrollments
 - Computes which lessons should have unlocked since last run
 - Sends email per student

This is more ops-heavy; you can skip it for v1 and still have drip working in UI.

Point is: **never let email failure break core logic.**

6. QUIZZES – SCHEMA

Goal:

- Each lesson can have an optional quiz.
- Quiz has questions; each question has multiple options; one correct.

- Student attempts are stored.

6.1 Tables

Run in Supabase:

```
create table if not exists quizzes (  
  id uuid primary key default gen_random_uuid(),  
  lesson_id uuid not null references lessons(id) on delete cascade,  
  title text,  
  created_at timestamptz default now()  
);
```

```
create unique index if not exists quizzes_lesson_id_idx  
on quizzes(lesson_id);
```

```
create table if not exists quiz_questions (  
  id uuid primary key default gen_random_uuid(),  
  quiz_id uuid not null references quizzes(id) on delete cascade,  
  prompt text not null,  
  order_index int not null default 0  
);
```

```
create index if not exists quiz_questions_quiz_id_idx  
on quiz_questions(quiz_id);
```

```
create table if not exists quiz_options (  
  id uuid primary key default gen_random_uuid(),  
  question_id uuid not null references quiz_questions(id) on delete  
cascade,  
  text text not null,  
  is_correct boolean default false  
);
```

```
create index if not exists quiz_options_question_id_idx  
on quiz_options(question_id);
```

```
create table if not exists quiz_attempts (  
  id uuid primary key default gen_random_uuid(),  
  quiz_id uuid not null references quizzes(id) on delete cascade,  
  student_id uuid not null references students(id) on delete  
cascade,
```

```

    score numeric(5,2),
    total_questions int,
    created_at timestamptz default now()
);

create index if not exists quiz_attempts_quiz_id_idx
on quiz_attempts(quiz_id);

create index if not exists quiz_attempts_student_id_idx
on quiz_attempts(student_id);

```

Simplified: one quiz per lesson. Good enough for v1.

7. QUIZ APIs – CREATOR SIDE

We'll build:

- `GET /api/creator/lessons/[lessonId]/quiz` – fetch quiz + questions + options
- `POST /api/creator/lessons/[lessonId]/quiz` – create/update quiz

7.1 GET quiz

`app/api/creator/lessons/[lessonId]/quiz/route.js`:

```

import { NextResponse } from "next/server";
import { getAuthUserFromRequest } from "@/lib/apiAuth";
import { getCreatorOrError } from "@/lib/creatorHelpers";

async function ensureLessonOwnedByCreator(supabase, lessonId,
creatorId) {
  const { data, error } = await supabase
    .from("lessons")
    .select("id, module:modules(course:courses(creator_id))")
    .eq("id", lessonId)
    .maybeSingle();

  if (error) {
    console.error("Quiz lesson fetch error:", error);
    return { error: "Failed to verify lesson", status: 500 };
  }
}

```

```

    }

    if (!data || data.module.course.creator_id !== creatorId) {
      return { error: "Lesson not found", status: 404 };
    }

    return { lesson: data };
  }

export async function GET(req, { params }) {
  try {
    const { lessonId } = params;
    const { user, error, status, supabase } = await
getAuthUserFromRequest(req);
    if (error) return NextResponse.json({ error }, { status });

    const { creator, error: creatorError, status: creatorStatus } =
      await getCreatorOrError(supabase, user.id);
    if (creatorError) {
      return NextResponse.json({ error: creatorError }, { status:
creatorStatus });
    }

    const { error: ownershipError, status: ownershipStatus } =
      await ensureLessonOwnedByCreator(supabase, lessonId,
creator.id);
    if (ownershipError) {
      return NextResponse.json(
        { error: ownershipError },
        { status: ownershipStatus }
      );
    }

    const { data: quiz, error: quizError } = await supabase
      .from("quizzes")
      .select("id, title")
      .eq("lesson_id", lessonId)
      .maybeSingle();

    if (quizError) {
      console.error("Quiz fetch error:", quizError);
      return NextResponse.json(

```

```

        { error: "Failed to fetch quiz" },
        { status: 500 }
    );
}

if (!quiz) {
    return NextResponse.json({ quiz: null, questions: [] });
}

const { data: questions, error: questionsError } = await
supabase
    .from("quiz_questions")
    .select("id, prompt, order_index")
    .eq("quiz_id", quiz.id)
    .order("order_index", { ascending: true });

if (questionsError) {
    console.error("Quiz questions error:", questionsError);
    return NextResponse.json(
        { error: "Failed to fetch questions" },
        { status: 500 }
    );
}

const questionIds = (questions || []).map((q) => q.id);

let options = [];
if (questionIds.length > 0) {
    const { data: optionsData, error: optionsError } = await
supabase
        .from("quiz_options")
        .select("id, question_id, text, is_correct")
        .in("question_id", questionIds);

    if (optionsError) {
        console.error("Quiz options error:", optionsError);
        return NextResponse.json(
            { error: "Failed to fetch options" },
            { status: 500 }
        );
    }
}

```

```

    options = optionsData || [];
  }

  const questionsWithOptions = (questions || []).map((q) => ({
    ...q,
    options: options.filter((o) => o.question_id === q.id),
  }));

  return NextResponse.json({ quiz, questions: questionsWithOptions
});
} catch (err) {
  console.error("GET creator lesson quiz error:", err);
  return NextResponse.json(
    { error: "Internal server error" },
    { status: 500 }
  );
}
}

```

7.2 POST quiz (simple full replace) [Inference]

You can choose to implement full CRUD; minimal version: “save quiz definition” replaces existing questions/options.

POST /api/creator/lessons/[lessonId]/quiz – body:

```

{
  "title": "Quiz title",
  "questions": [
    {
      "prompt": "Question 1?",
      "options": [
        { "text": "A", "is_correct": false },
        { "text": "B", "is_correct": true }
      ]
    }
  ]
}

```

Implementation outline:

- Check ownership as above.

- Upsert quiz row (`lesson_id`).
- Delete existing questions/options for that quiz.
- Insert new questions/options.

You already know the patterns; follow same try/catch + logging + safe defaults.

8. QUIZ APIs – STUDENT SIDE (ATTEMPTS)

We'll implement:

- `GET /api/learn/lessons/[lessonId]/quiz` – retrieve quiz to show in learner UI.
- `POST /api/learn/lessons/[lessonId]/quiz/attempt` – grade + store attempt.

8.1 GET quiz for student

Very similar to creator GET, but:

- Ensure student is enrolled in the course.
- Don't expose `is_correct` in options.

You can reuse logic, just filter out `is_correct` before returning.

8.2 POST attempt

Flow:

- Auth → student profile
- Check that student is enrolled in corresponding course.
- Body: `{ answers: { [questionId]: optionId } }`
- Calculate score as `correct / total_questions`.
- Insert into `quiz_attempts`.

Make sure:

- If quiz or questions missing → return 400 with clear error.

- Don't crash on invalid IDs; just fail the request cleanly.
-

9. QUIZ UI – CREATOR EDITOR [Inference]

In `EditCoursePage` → `ModuleItem` → `LessonItem`, let creator open a “Quiz editor” for each lesson.

Minimal approach:

- Add a button under each lesson: “Edit quiz”.
- Clicking opens a modal or expanded area with:
 - Quiz title
 - List of questions
 - Each question: prompt + options + radio button to mark correct.
- On save → POST to `/api/creator/lessons/[lessonId]/quiz`.

Because wiring full UI is verbose, you can start with:

- Hardcoded 1 question / 2 options per lesson.
- Then gradually generalize.

Key point:

all network calls wrapped in try/catch, with local `error` + `saving` flags so you never blow up your whole editor because the quiz API returned a 400.

10. QUIZ UI – STUDENT LEARN PAGE [Inference]

In `LessonView`, if lesson has a quiz (presence determined via `GET /api/learn/lessons/[lessonId]/quiz`):

- Fetch quiz when lesson selected.
- Render:
 - question prompt
 - radio buttons for options

- On submit:
 - POST attempt
 - Show score & simple feedback.

Important:

If quiz API fails → show “Quiz failed to load” and do nothing else. **Do NOT crash the whole learn page.**

11. Phase 7 Hygiene Checklist

You're done with Phase 7 when:

Drip

- `courses.drip_enabled` toggling works in editor.
- `lessons.drip_delay_days` stored correctly.
- `/learn/[courseId]`:
 - shows locked lessons as disabled.
 - auto-selects first unlocked lesson.
 - doesn't crash if 0 unlocked lessons.
- Enrollment date changes unlock behavior predictably (enroll a test user and change system time or delay to verify logic).

Email

- App runs fine even with no email env vars set.
- Enrollment triggers `sendEmail()` without throwing.
- Errors in email sending are logged, not fatal.

Quizzes

- Quiz schema exists in Supabase.
- Creator can:
 - create quiz for a lesson.

- edit and overwrite quiz.
- Student can:
 - load a lesson and see its quiz (if exists).
 - submit answers.
 - API stores attempts without duplicates / crashes.
- No direct exposure of `is_correct` to students in the GET quiz API.

General

- Every new API:
 - handles auth properly.
 - uses `try/catch`.
 - returns structured JSON error messages.
- Frontend:
 - never assumes presence of quiz/drip fields.
 - always guards rendering on `loading/user/creator` checks.

At this point you have:

- Core SaaS
- Billing
- Analytics
- Drip content
- Quizzes
- Email hooks

That's enough to ship a legit v1 and start worrying about polish, performance, and security hardening instead of basic functionality.